

University of
Sheffield

Implementing a Password Manager

A Vault Broker Design for Limiting
Credential Exposure After Unlock

Rohith Halappa Chavan

Supervisor: Dr. Prosanta Gope

A report submitted in fulfilment of the requirements
for the degree of MSc in Cybersecurity and Artificial Intelligence

in the

School of Computer Science

September 16, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Rohith Halappa Chavan

Signature: Rohith Halappa Chavan

Date: September 16, 2026

Abstract

A password manager must protect passwords both while they are stored and while they are being used. This project examines how to control access to saved passwords after a user unlocks the vault. A separate program, called a vault broker, opens the vault and handles sensitive actions. The main window normally receives only account summaries. Before a sensitive action can go ahead, the broker asks for approval. If approval succeeds, it issues a temporary permission token. Each token allows one action on one account record and can be used only once.

The prototype runs locally on Windows. It is written in Python and uses PySide6 for its windows and controls. Argon2id converts the master passphrase into a key that unlocks the vault key. AES-256-GCM encrypts the saved information and checks whether it has been changed. Windows Hello is part of the approval process for sensitive actions. The application can also generate passwords, make encrypted backups, show a password temporarily, clear copied passwords from the clipboard and lock the vault.

The evaluation uses source-code checks, recorded automated tests and a simplified Tamarin model of the permission rules. The recorded test run contains 305 passed tests and seven skipped tests, with no failures or errors. Six separate Tamarin runs confirm that an intended sequence of actions is possible and that five stated rules hold within the model. These results show how the request controls behave in the cases checked. They do not prove that the whole application is safe from every kind of malware.

The project shows how a broker can restrict what the main window receives through normal requests. This protection has limits. Passwords can be present in the broker's memory, and other software may capture a password after it is copied or shown. Stronger separation from other software, more precise approval checks and further testing with users are needed before making broader claims about security.

Contents

Abstract **ii**

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Aim, Research Question and Objectives . . . 2
1.4 Project Scope . . . 3
1.5 Report Structure . . . 3

2 Literature Review **4**
2.1 Password Manager Approaches . . . 4
2.2 Vault Encryption and Master Passphrase Protection . . . 5
2.3 Credential Exposure During Use . . . 6
2.4 User Approval and Usability . . . 7
2.5 Comparison of Existing Approaches . . . 8
2.6 Focus of This Project . . . 9

3 Requirements and Development Approach **10**
3.1 Functional Requirements . . . 10
3.2 Security and Usability Requirements . . . 11
3.3 Threat Model and Assumptions . . . 12
3.4 Development and Testing Approach . . . 12
3.5 Project Planning and Ethical Considerations . . . 13

4 System Design **14**
4.1 Architecture Overview . . . 14
4.2 Vault Encryption and Key Handling . . . 15
4.3 User Approval and Limited Permissions . . . 17
4.4 Credential Use and Locking . . . 19
4.5 Backups and Audit Records . . . 19
4.6 Design Decisions and Trade-offs . . . 20

5 Implementation **21**
5.1 Project Structure and Tools 21
5.2 Encrypted Vault and Credential Operations 22
5.3 Broker Communication and Permission Checks 22
5.4 User Interface and Windows Hello 23
5.5 Clipboard, Locking and Audit Logs 28

6 Testing and Evaluation **29**
6.1 Evaluation Approach 29
6.2 Functional and Security Test Results 29
6.3 Tamarin Model Results 31
6.4 Evaluation Limitations 33

7 Discussion and Conclusion **34**
7.1 Answer to the Research Question 34
7.2 Comparison with Related Work 34
7.3 Project Limitations and Lessons Learned 35
7.4 Future Work 36
7.5 Conclusion 37

Appendices **41**

A Installation and User Guide **41**
A.1 Installation and Running the Application 41
A.2 Creating and Using a Vault 41
A.3 Backup and Restore 42

B Additional Test Evidence **43**
B.1 Detailed Test Results 43
B.2 Code Quality and Dependency Checks 44
B.3 Environment and Tool Versions 45

C Formal Model and Proof Evidence **46**
C.1 Tamarin Model 46
C.2 Proof Commands and Tool Versions 49
C.3 Individual Proof Results 50

List of Figures

4.1 Vault Broker Architecture 14
4.2 Key Derivation and Vault Encryption 16
4.3 Approval and Single-Use Permission Flow 18

5.1 Vault Creation and Unlock Screens 24
5.2 Main Credential List 25
5.3 Sensitive Action Approval and Windows Hello 26
5.4 Password Generation and Credential Editing 27

List of Tables

2.1 Selected Approaches and Their Relevance to This Project 8

3.1 Functional Requirements 10
3.2 Security and Usability Requirements 11

5.1 Main Implementation Areas 21

6.1 Summary of Recorded Implementation Checks 30
6.2 Recorded Tamarin Results 32
6.3 Main Claims, Existing Evidence and Limits 33

B.1 Recorded Test Cases by Module 43
B.2 Recorded Test and Tool Information 45

C.1 Individual Targeted Proof Runs 50

Chapter 1

Introduction

1.1 Background

People often have separate online accounts for email, banking, shopping, study and work. Many of these accounts still rely on passwords. Remembering a different password for every account can be difficult. Reusing a password is easier, but it creates a problem: if that password is stolen from one account, an attacker may be able to use it elsewhere. Researchers have looked for ways to reduce this problem for many years. Halderman, Waters and Felten, for example, studied how to generate different passwords from a small amount of information that the user remembers [1].

A password manager helps someone store account details and generate passwords. The saved collection of details is usually called a *vault*. Encryption changes the saved data into a form that cannot normally be read without the correct key. The master passphrase helps the application unlock that key. Protecting the saved file is part of the job. Once the vault is open, the application also needs rules about when a password can be shown or copied, and which part of the program can access it.

Earlier studies explain why these rules matter. Oesch and Ruoti checked password generation, storage and autofill in thirteen managers and found weaknesses in these areas [2]. Tapas used a second device to help protect passwords both in storage and during use [3]. These studies show that an encryption algorithm alone cannot address every problem. The application must also handle passwords carefully during everyday tasks.

1.2 Problem Statement

An unlocked vault lets the application work with its saved records. The access given to the main window is therefore important. If that window can ask for every password at once, one unwanted request could expose the whole collection. If it receives only account summaries and must obtain approval for each sensitive action, there are more checks before a password can be released. This project investigates that approach.

The design separates the main window from the part of the application that handles the vault. The main window is the graphical user interface, or GUI: the part the user sees and interacts with. The part that handles the vault is called the *vault broker*. It runs in

its own process, separately from the main window. While the vault is open, the broker holds the vault key and checks requests from the GUI. A request is a message asking the broker to do something, such as copy one password.

The attacker considered in this project controls the requests sent by the GUI. The broker and the operating system are assumed to continue working correctly. This assumption allows the project to examine the request rules. It leaves out attacks that read or change the broker's memory. Having two processes on the same computer does not, by itself, prevent all attacks between them.

The attacker also differs from those considered in some related work. Scarlata and colleagues examine attacks by malicious cloud servers [4]. This application has no cloud service, so their attacks and results do not apply to it directly. A design needs to be assessed against the attacker it is meant to resist. Chapter 3 sets out what this project's attacker can do and which parts of the application must remain trusted.

1.3 Aim, Research Question and Objectives

The aim is to build and evaluate a local password manager that controls access to saved credentials after the vault is unlocked. Here, *credentials* means account details, such as a username and password. The research question is:

> To what extent can a separate vault broker, using short-lived, single-use permissions, limit the credentials a compromised GUI can obtain through the broker's normal request interface?

In this report, a compromised GUI means a main window whose requests are controlled by an attacker. The temporary permission mentioned in the question is a token issued after approval. This kind of token is called a *capability*. It permits one action on one record. For example, permission to copy an email password should not also let the caller delete a different account. The project checks whether these rules are followed. It does not try to calculate a percentage reduction across all possible malware attacks.

The objectives are to:

1. review related password-manager designs and identify which parts each design needs to trust;
2. build an encrypted local vault whose operations are separate from the main window;
3. require approval for sensitive actions, then check the action allowed by a token, its expiry time and whether it has already been used;
4. use automated tests and code checks to examine selected features and security rules;
5. model the approval and permission rules in Tamarin and compare the model with the application;
6. explain the results, the remaining limits and useful next steps.

The main contribution is a working Python prototype on Windows and an examination of its broker request rules. Approval and restrictions on password release have already

been explored in earlier work, including Tapas [3]. This project applies those ideas by separating vault operations from the main window and using temporary permissions for sensitive actions. The available tests and model help explain how these rules behave and where their protection ends under the stated assumptions.

1.4 Project Scope

The final prototype runs locally on Windows 11 with Windows Hello configured. Its features include creating and unlocking vaults, managing accounts, generating passwords, approved copying and revealing, encrypted backups and locking. This report examines version 0.8.0. The application is supplied as Python source code with installation instructions; a finished Windows installer is not included.

Cloud synchronisation, browser autofill, password sharing and master-passphrase recovery are outside the application. Windows Hello approves sensitive actions. It does not replace the master passphrase or hold a recovery copy of the vault key. An encrypted backup can help when the working file is lost, but the correct master passphrase is still needed to open it.

The final prototype also has no AI detection feature. Research on honey vaults, which hold believable decoy data, is discussed as a different approach [5]. Honey vaults are not a feature of this application. Keeping the research discussion separate from the implemented features makes clear what was actually built.

The available evidence consists of source code, a recorded automated test run, code-quality logs and six Tamarin proof logs. There is no formal user study, measured comparison with commercial password managers or completed code-coverage report. The claims in the later chapters are kept within these limits.

1.5 Report Structure

Chapter 2 reviews work on password storage, password release, approval and usability. Chapter 3 explains the requirements, assumptions about the attacker and development approach. Chapter 4 describes the design, followed by its implementation in Chapter 5. The tests and model results are presented in Chapter 6. Chapter 7 answers the research question, discusses the main lessons and sets out future work. The appendices provide the user guide, more detailed test results and formal-model evidence.

Chapter 2

Literature Review

2.1 Password Manager Approaches

Password managers store and provide access to account details in different ways. A local manager keeps an encrypted vault on the user's device. A cloud-based manager uses an online service to store data or keep copies up to date across devices. A store-less manager creates an account's password when it is needed, rather than saving it in a normal password database. Each approach affects security, convenience and recovery in different ways. Jubur, Shrestha and Saxena review these choices by looking at security, privacy, ease of use and the practical effort involved [6].

Password Multiplier is an early design that creates passwords instead of storing them. Halderman and colleagues use a remembered secret together with other information to produce a different password for each website [1]. This reduces how much the user needs to remember. To produce the same password later, however, the system needs the same inputs. It also has to deal with password changes and the different password rules set by websites. The paper helps explain this approach, but its older algorithms and measured times are not suitable settings for the present project.

HIPPO is a more recent store-less design. A client program works with a server to produce account passwords without saving them in a conventional vault [7]. The protocol uses an oblivious pseudo-random function: a method that combines information from the two sides while limiting what either side learns about the other's input. Its security claims depend on which parts are trusted and which parts an attacker can compromise. The relevant security argument assumes that the client can be trusted. It therefore does not establish protection when an attacker already controls the client.

These designs face different security problems. A local manager has to protect both the vault file and the program that opens it. A store-less manager must produce the correct passwords again and protect the inputs used to make them. A cloud manager must also protect its online service and the process of keeping copies up to date. This project uses local encrypted storage to focus on what happens after the vault is unlocked. Although the application has no cloud server of its own, an attacker could still steal its vault file or attack the computer.

Keeping the vault locally also puts more responsibility on the user. The user needs to know where the file is and keep backups. Passwords do not automatically become available on another device. These limits affect everyday use, so the comparison needs to consider both the parts that must be trusted and the work left to the user. Local storage is a choice for this project's focus, not a claim that it suits everyone best.

2.2 Vault Encryption and Master Passphrase Protection

The strength of the master passphrase is one part of protecting a saved vault. A key derivation function, usually shortened to KDF, turns the passphrase into a key. It also uses a random value called a salt. Different salts give different results even when two users choose the same passphrase. The KDF requires work for every attempt, making repeated guesses more costly [8]. This helps slow guessing, but it does not turn a predictable passphrase into something as strong as a randomly generated encryption key.

RFC 9106 describes Argon2id, the KDF used in this project [8]. It needs memory as well as processing time. The key it produces unlocks a separate vault key. The vault settings record how much memory and work to use, and the application checks those settings before using them. The exact values appear in Chapter 4. Increasing the cost affects both an attacker trying guesses and the user waiting to open the vault. Measurements on the intended hardware would help show whether that waiting time is acceptable.

Encryption also needs a way to detect changes to the data. AES-GCM provides *authenticated encryption*. Alongside the encrypted information, it produces a tag that is checked when the information is read. The check can include related information that remains unencrypted, known as associated data; a record number is one example. Each encryption also uses a short value called a nonce. GCM requires a different nonce for every encryption made with the same key [9]. These details need to be used correctly. Choosing the algorithm alone is not enough.

Bellare and Shea study stronger requirements for encryption based on passwords [10]. They consider attacks in which an accept-or-reject response helps an attacker reduce the set of possible keys. They also examine whether the same encrypted data could be accepted under more than one candidate key. Their construction addresses particular requirements arising from these problems. The paper is useful for understanding the limits of an encryption claim. This prototype uses a KDF and AES-GCM, but it does not implement the complete construction that Bellare and Shea prove secure.

Scarlata and colleagues study three password-manager providers in a setting where the cloud server is malicious [4]. They find problems in both the cryptography and the surrounding application behaviour. Their work draws attention to record information, checks for changes and support for older formats. The attacks involve remote services, so they do not demonstrate attacks on this local prototype. The findings discussed here come from the paper on three providers and are kept separate from those in its later expanded version.

Claims about encryption also need careful checking in prototype papers. Pandare and colleagues describe a design using SHA-256 and AES [11]. The stated bit lengths of two algorithms cannot simply be added to calculate a stronger security level. A hash cannot be

decrypted in the way that encrypted data can. This example shows why the report needs to explain what each algorithm does. It should be possible to follow which key protects which information, without relying on a broad description such as “hybrid encryption”.

For this project, the storage design therefore distinguishes the master passphrase, the key derived from it, the randomly generated vault key and the encrypted records. Each has its own job. The design also needs to explain when data is accepted or rejected. Strong encryption cannot make up for every error in the software around it.

2.3 Credential Exposure During Use

Eventually, a saved password needs to be used. It might be displayed, copied or entered into another program. Somewhere in that process, the password must be readable. This raises several questions: which part of the system receives it, does its release match the user’s request, and does that request expose only one password or make other passwords accessible too? These questions are central to this project.

Tapas is a close comparison because it gives a computer and a smartphone separate roles in protecting and accessing passwords [3]. Its security discussion explains that malware on the computer may still capture a password when that password is used. Under the design’s stated assumptions, the aim is to protect the other saved passwords. The idea of limiting exposure during use already exists in this work. This project builds on that idea through a local broker implementation and checks of its behaviour.

The separation used here is different from the one in Tapas. Tapas relies on two devices; this application runs two processes on the same Windows computer. A process is a running program. The broker provides a separate place to check requests, but it shares the operating system with the main window. Two processes should not be assumed to give the same protection as two devices. This distinction affects what an attacker would need to control to reach the saved secrets.

Li and colleagues examined five web-based password managers and found serious weaknesses, including problems with authentication and the handling of web requests [12]. Those findings apply to the products and versions they tested at the time. They do not show that today’s versions still have the same weaknesses. For this project, the relevant lesson is that requests for a password deserve as much attention as the vault’s encryption. The broker interface needs checks of requests that should succeed and requests that should be refused.

Oesch and Ruoti examine password generation, storage and autofill [2]. Their study shows why progress in one area cannot settle the security of the others. For example, a manager could create a strong password but later expose it through unsafe autofill behaviour. This application uses copy and reveal actions instead of autofill. That changes how a password leaves the manager, but the clipboard and main window can still expose it.

In Leaky Autofill, the authors test hidden form fields across thirty password managers [13]. They report cases where sensitive information is filled into fields that do not match what the user expects to see. The result shows the importance of checking where

information goes. The paper's leakage rates describe its own test cases. They are not results for this application, which neither examines web forms nor fills them automatically.

AutoFail follows information as it moves between web pages, Android's autofill system and password managers [14]. Its evaluation covers nine managers and five browsers. Some problems arise because useful context is lost or understood differently as information passes between these parts. A component may then release a password without all the information it needs to make that decision. For the broker design, this supports tying a permission to a particular record and action. It also makes the missing details worth examining: the current token checks do not cover the record's version or every detail of a requested change.

An approval prompt cannot resolve all of these risks. The request could be unclear, its destination could be unsafe, or another program could capture the password once it has been released. The broker addresses a narrower problem. Before accepting a sensitive request, it checks the record, the action and the temporary permission. It cannot establish that every destination is safe or that every user decision is correct.

2.4 User Approval and Usability

Asking for approval adds a check, but it also takes the user's time and attention. The prompt must explain what is being requested, and repeated prompts may become tiring. Jubur, Shrestha and Saxena discuss how security, ease of use and people's willingness to adopt a manager are connected [6]. An extra check does not itself show that people will understand the request or choose the intended response.

Tapas includes a study with thirty participants and a smaller follow-up study [3]. This provides evidence about using the design as well as a description of how it works. The findings still belong to those participants, tasks and the two-device setup. They cannot tell us how people would respond to the prompts in this Windows application. Some interface behaviour is covered by this project's automated checks, but there is no comparable participant study.

The later HIPPO study compares its store-less manager with password-only authentication using twenty-five university participants [15]. The comparison is against logging in with a password alone, rather than against another conventional password manager. Participants complete the tasks in a fixed order during a short laboratory session. This differs from using an application in daily life over several months. Their views about trust and security help describe the experience of using HIPPO, but those views do not prove the security of its protocol.

Windows Hello needs the same careful distinction in this report. Successful verification means that the Windows check has succeeded. It does not show whether the user understood the requested action or approved it out of habit. Checking successful verification and cancellation can show how the program handles those outcomes. Understanding whether people become tired of the prompts would require a study with users.

Halim and colleagues examine factors affecting adoption through responses from 136 people in the Jabodetabek region [16]. Their questionnaire helps explore perceived

usefulness and trust. It does not measure whether those users experienced fewer stolen accounts. This is why the report separates different types of evidence. A person saying that a manager feels safer cannot replace an attack test. Equally, passing an attack test does not show that people find the interface easy to use.

Rajarajan and colleagues study a graphical master password based on choosing images [17]. Helping people remember their password is a worthwhile aim. However, turning a limited set of image choices into a long string does not increase the number of original choices. The difficulty of guessing depends on that choice space, not only on the resulting string's length. This application uses a text passphrase and does not implement this graphical method.

The approval workflow in this project should show the action and account title, provide a clear way to cancel, and refuse access when verification fails. The program can be checked and tested for these behaviours. Whether a wider group of users would find the workflow comfortable is still unknown. The evaluation keeps that question open rather than treating working prompts as proof of usability.

2.5 Comparison of Existing Approaches

Table 2.1 brings together the approaches most relevant to this work. It compares their focus and the differences that matter for the broker design. The table does not rank the systems. They consider different attackers, devices and tasks, so a single combined security score would hide those differences.

Table 2.1: Selected Approaches and Their Relevance to This Project

Approach	Main focus	Important difference or limit
Password Multiplier [1]	Generating a password for each site	Older design; handles password changes differently from a stored vault.
Tapas [3]	Protecting and releasing passwords using two devices	Two devices provide a different separation from two local processes.
HIPPO [7]	Generating passwords with an online protocol	Uses a server and a trusted client; does not study this broker interface.
Autofill studies [13, 14]	Mistakes in where and when passwords are filled	Study web and Android systems, not Windows copying and revealing.
Cloud analysis [4]	Attacks by malicious servers	Studies a remote-service attacker absent from this prototype.
This project	Checking local broker requests after unlock	Assumes a trusted broker and operating system.

Passkeys offer another way to authenticate to an account. Reittinger and Pernul study adoption and use on one platform over six months [18]. In the reported population, 5,057 people chose an authentication method, and 2,716 made at least one authentication attempt during the later usage phase. The latter group includes both passkey users and users of passwords with two-factor authentication. Following actual use over time is valuable, although the findings still depend on that service and its users. The study gives

a reason to consider alternatives to passwords. This dissertation focuses on accounts that continue to use passwords.

Honey vaults take a different approach to password guessing. Cheng and colleagues create believable decoy vaults and examine whether attackers can identify them or check their guesses online [5]. The method learns patterns from data. Its results depend on those patterns and the conditions of the attack, including restrictions on online checks. This application has neither a decoy vault nor a learned model. Adding that protection would need its own design and evaluation; displaying made-up account records after a failed unlock would not be enough.

These comparisons also show why evidence needs to match the question being asked. A screenshot shows what a screen looks like. A test checks a selected behaviour. A user study examines people's experience, while a formal proof checks rules within a model. Each is useful for a different purpose. This report uses the evidence for the questions it can answer and takes account of each paper's methods and limits.

2.6 Focus of This Project

The literature collection contains thirty-one papers. All were checked for their purpose and relevance. Papers most closely related to vault storage, password release, approval and trust were read in more detail. This chapter groups them by topic to connect the discussion directly to the research question. Papers with less relevance receive less space, since the comparison depends on how closely each paper addresses the project's focus rather than on the number of citations.

The project focuses on a local vault managed through a broker. For ordinary browsing, the main window receives account summaries. Sensitive actions require approval and a temporary token for one action on one record. This gives the evaluation specific questions to answer. Does a list response include a password? Can a token allow the wrong action or be used more than once? What happens when the token expires or the vault is locked?

Earlier research already examines limited password release and risks during use. The question here is how those ideas work in a local Windows broker with temporary, single-use permissions for individual records and actions. This is a focused implementation study, not a claim that similar designs do not exist. The source code, tests and formal model allow the chosen request rules to be examined. They also help explain the limits of the protection those rules provide.

Chapter 3

Requirements and Development Approach

3.1 Functional Requirements

The requirements set out what the final prototype should do and provide a basis for checking its main tasks. The central task is to keep account details in an encrypted local vault and give the user access to a selected account when needed. The requirements describe the final project scope. They do not establish when each requirement was agreed or whether it was approved before a particular development decision.

Each requirement in Table 3.1 has a short identifier, which the evaluation uses to connect it with the supporting evidence. Creating a vault and unlocking one are separate tasks. Creation sets up a new vault key and master passphrase. Unlocking checks the passphrase for an existing vault. The user can work with account records once the vault is open.

Table 3.1: Functional Requirements

ID	Requirement
FR1	Create an encrypted vault and unlock it using its master passphrase.
FR2	Add, list, search, view, edit and delete account records.
FR3	Generate passwords and support approved copying and temporary display.
FR4	Ask for user approval through the broker before allowing sensitive actions.
FR5	Lock the active vault and cancel all unused permission tokens.
FR6	Create an encrypted backup without overwriting an existing file.
FR7	Show clear feedback when input, approval, storage or a request fails.

An account record contains a title, website, username, password and notes. The normal list needs enough detail for the user to recognise and choose an account. Passwords and notes are not needed for that task, so the broker returns a summary of each record. These summaries can still reveal personal information. A title or username, for instance, may show which services someone uses. The aim is to keep passwords out of routine replies, while still showing the information needed to choose an account.

The user is responsible for the locally stored vault file, so the application needs a way to make a backup safely. The copy must remain encrypted, and making it must not accidentally overwrite another file. The user also needs clear guidance about the master passphrase required to open the backup.

3.2 Security and Usability Requirements

A security requirement should describe behaviour that can be checked. Calling storage "secure" says little about what should happen if someone changes the saved data. A clearer requirement says that a record must be rejected when its encryption tag is invalid. A list reply can also be checked to see whether it contains passwords. Table 3.2 describes the main security and usability requirements through checks of this kind.

Table 3.2: Security and Usability Requirements

ID	Requirement
SR1	Encrypt saved account details, check for changes to them, and use the master passphrase to derive the key that unlocks the vault key.
SR2	Keep vault operations in the broker and leave passwords out of normal list and search responses.
SR3	Issue tokens only after approval. Limit each token to one record, one action, a short lifetime and one use.
SR4	Reject invalid or unsupported requests and deny sensitive actions when approval fails.
SR5	Close the broker session on lock and apply time limits to copied and displayed passwords.
SR6	Keep passwords, master passphrases and permission tokens out of audit records.
UR1	Show recognisable account details and clear action prompts, with feedback when an action is denied.
UR2	Explain the limits of backups, Windows Hello and passphrase recovery in the user guide.

An approval prompt should identify both the action and the account. The user can then check what they are being asked to allow. Cancelling should have a clear outcome too: the application must not report success after the user cancels approval. These behaviours can be inspected in the interface and checked through automated tests.

Whether people find the prompts easy to use is a separate question. No participant study has measured how quickly users understand them or how repeated requests affect their decisions. The interface and its tests show how the design aims to meet UR1, but they do not fully show how well it works for users. This gap matters because the design depends on approval for sensitive actions.

Recovery limits must also be explained clearly. Windows Hello and an encrypted backup cannot reset a forgotten master passphrase. If the guide does not make this clear, someone may expect another way to open the vault and lose access to their records. This guidance is part of the design because it affects how the user can safely use the application.

3.3 Threat Model and Assumptions

A threat model sets out what an attacker can do and which parts of the system must keep working correctly. The main attacker here controls requests sent through the normal connection between the GUI and the broker. It can choose operations, send badly formed messages, ask for permissions and try to use a token again. It can also read the information returned to the GUI. If a reveal is approved, the returned information includes that password. A password copied to the system clipboard may also be read there.

The broker, its approval decision and the operating system are assumed to remain trusted. The attacker in this model cannot read the broker's memory, change its code or insert code into its process. These limits define what "compromised GUI" means in this report. They make it possible to study the request controls without claiming to cover every attack by software running under the same Windows account.

Windows uses permissions and access rights to control access to processes [19]. Running the broker separately helps divide responsibilities, but a separate process is not automatically protected from every other process. An attacker who can inspect the broker directly can bypass the request interface. In that case, the main assumption no longer holds. Protecting against that attacker would require further operating-system controls and testing.

The saved vault file presents another problem: someone may steal or alter it. The master-passphrase derivation and encryption are intended to protect this stored data. They make records harder to read without the key and allow certain changes to be detected. They cannot prevent deletion of the only copy, and a predictable passphrase can still be guessed. Replacing the entire file with an older valid copy is another limit, discussed in Chapter 4.

These assumptions limit what a successful check tells us. The broker may reject a token's second use even if another program has already captured the password from its first approved copy. Windows may verify the user without knowing whether that person understood the request. Locking takes effect only once the broker receives the lock request. Some lock requests start in the GUI, so an attacker controlling that GUI could prevent them from being sent. The evaluation considers these situations separately.

3.4 Development and Testing Approach

The project uses a build-and-test approach. The required tasks are implemented, responsibilities are split into modules, and automated tests check selected behaviours. The final source separates encryption helpers, storage, broker communication, security utilities and the main window. This allows a permission rule to be tested without opening every screen. Other tests check how the parts work together.

Tests need to cover successful actions as well as requests that should be rejected. Saving a record and reopening it checks normal use. Trying to open encrypted data after it has been altered checks a different condition. Likewise, using a token successfully once does not show that a second use will fail. Tests must also try the wrong record, the

wrong action and an expired token. These cases address the rules that matter most to the research question.

Some tests control the clock or use a substitute approval result to make the same conditions easier to repeat. A test can move the clock past a token's expiry without waiting for real time to pass. A substitute approval result can return success or cancellation, allowing both outcomes to be checked. These methods are useful, but the relevant behaviour still needs to be tested on real Windows hardware.

Tamarin checks a different part of the design. It works with simplified rules for approval, token use, expiry and locking. Comparing those rules with the source code reveals details that the model leaves out. The automated tests and the model therefore provide different kinds of evidence. A large number of tests or a verified model, taken alone, cannot show that every part of the application is secure.

3.5 Project Planning and Ethical Considerations

The report follows the main sequence of work: understanding the problem, setting the scope, explaining the design, describing the implementation and assessing the results. A complete dated development diary and reliable records of the hours spent on each task are not available. The planning discussion therefore covers the selected scope and completed work, rather than a week-by-week history.

A focused scope makes the results easier to assess. Cloud synchronisation, browser integration and recovery would each introduce further security questions and tests. They are outside the final prototype. The evaluation instead concentrates on the existing functional tests and permission model. It also identifies the user testing, hardware checks and performance measurements that are still needed.

The tests use temporary vaults and demonstration credentials. The report includes no real account secrets or personal information from study participants. No participant study was conducted, and no approval for such a study is claimed. Any later study would need the relevant university approval before collecting data. The report also distinguishes between a feature found in the source code, a recorded test result and an idea for future work. Each provides a different kind of information about the project.

Chapter 4

System Design

4.1 Architecture Overview

The application runs in two main processes. The GUI displays the account list and lets the user choose what to do. The vault broker opens the vault, holds the keys while it is open and carries out the actions it allows. Figure 4.1 shows how these parts work together. The main window asks the broker to perform each operation. It does not hold a `VaultSession`, which is the program object needed to work directly with an unlocked vault.

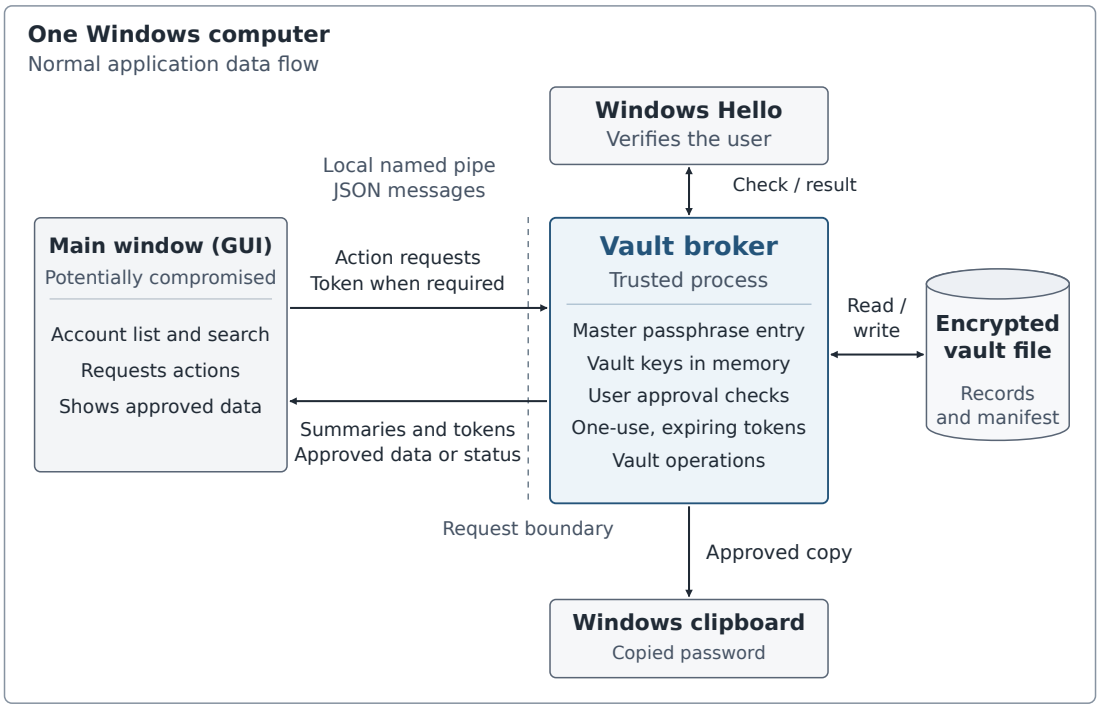


Figure 4.1: Vault Broker Architecture

The broker checks every request, even when it comes from the application's own GUI. It checks the operation name, supplied values and whether the vault is open. It also checks whether the action needs a permission token. Keeping these decisions in one place avoids relying on the screen to enforce them. For example, disabling a button can guide the user, but someone could send the same request without pressing it. The broker still has to enforce the rule.

For an ordinary list request, the broker returns the record identifier, title, website, username and version. The record identifier distinguishes one saved account from another. The response leaves out passwords and notes. Copying a password, revealing it and viewing the other record details each require a separate approved action. There is no interface operation for exporting all passwords in readable form. Ordinary requests therefore return less information, although account titles and usernames can still be seen.

Both processes run on the same computer, and the broker's confirmation windows appear on the normal Windows desktop. The broker is not a separate device. The design therefore relies on the broker's code, its memory and the Windows environment remaining trusted. Further controls would be needed for stronger protection from other software. This differs from Tapas, which uses two devices [3]. Both approaches separate password handling from the main interface, but one uses processes on the same computer and the other uses separate devices.

4.2 Vault Encryption and Key Handling

The keys have different jobs, as shown in Figure 4.2. To create a vault, the application generates a random 256-bit vault key, a 16-byte salt for Argon2 and a 16-byte vault identifier. Argon2id uses the master passphrase to produce another 256-bit key. This second key is called the *wrapping key*: it encrypts, or wraps, the vault key. It is not used directly to encrypt each saved account record.

When the user opens the vault later, the application reads the saved salt and settings and repeats the passphrase calculation. It uses the result to try to decrypt the stored vault key and check its authentication tag. The wrong passphrase cannot provide a usable vault key. This keeps the user's passphrase separate from the random key that protects the records.

Argon2id is configured with a time cost of 3, memory use of 65,536 KiB, parallelism of 1 and a 32-byte output, using version 19. The memory setting is equivalent to 64 MiB. Time cost controls the repeated work done by the function. Parallelism controls how many processing lanes it uses to divide that work. RFC 9106 explains Argon2 and recommends settings [8]. The application's parallelism value differs from the RFC's 64 MiB profile, so the configuration does not match that profile exactly. This report does not include measurements of unlock time on different machines.

The application checks saved settings before using them. Accepted values range from 19 MiB to 1 GiB for memory, 2 to 10 for time cost and 1 to 4 for parallelism. It rejects values of the wrong type, unsupported versions and incorrect output lengths. These checks reject some unsuitable or excessive requests. Even an accepted setting may place too much demand on a smaller machine, so the limits do not guarantee good performance.

HKDF-SHA-256 uses the vault key to derive two further keys. One protects the account records. The other protects the manifest, a list of what the vault should contain. HKDF allows labels to distinguish keys intended for different uses [20]. The application uses the vault identifier as the salt and a fixed purpose label for each output. All records in a vault share its record key; the application does not derive a different key for each account.

AES-256-GCM encrypts each record using a fresh random 12-byte nonce and a 16-byte authentication tag. A nonce is a short value used for that particular encryption, and the tag is checked when the record is read. The encryption also uses *associated data* to link the record to its vault identifier, record identifier and version. This helps detect a record used where a different one was expected. GCM requires a nonce never to be reused with the same key [9]. Generating nonces with secure randomness makes an accidental repeat unlikely, but does not make it mathematically impossible.

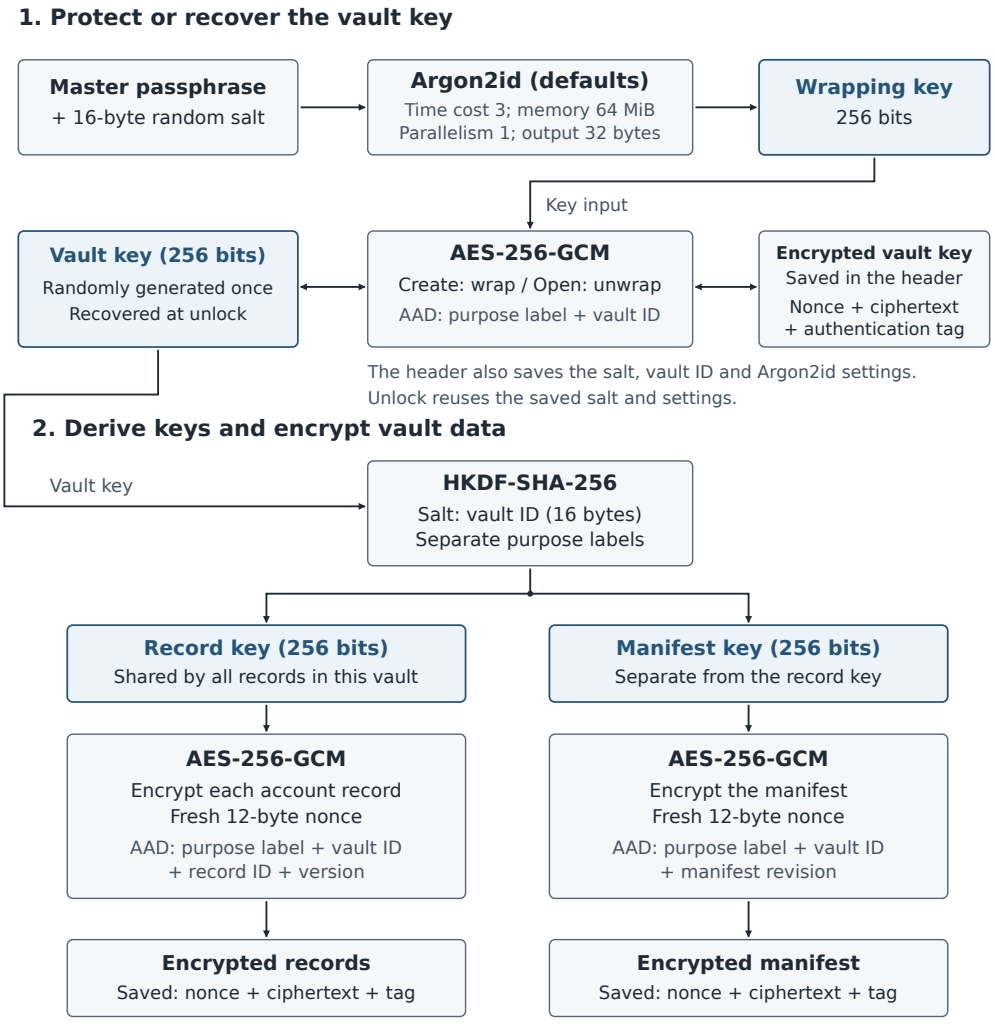


Figure 4.2: Key Derivation and Vault Encryption. Here, recovering the vault key means decrypting it with the correct master passphrase; it does not mean recovering a forgotten passphrase.

The encrypted *manifest* records what should be in the vault. It stores record identifiers, versions, digests of the encrypted data and a revision number. A digest is a fixed-size value calculated from data. Here, it helps check whether the data matches what is expected. The manifest lets the application check the collection as a whole. If someone removes a record without changing the current manifest, for example, the records no longer match the list. It can also help detect an individual record replaced with an older version.

The same check cannot always detect an older copy of the entire vault because the old records may still match the old manifest. The application has no separate trusted record of the latest revision to compare against, so it may accept the older file. This is the limit on detecting a complete *rollback*, where the vault returns to an earlier saved state. The limit applies to both storage checks and backup restoration.

The keys remain in the broker while the session is open. Closing the session removes the application's active references to them. However, Python or its libraries may have created other copies in memory. Removing a reference does not show that every copy has been overwritten. The design keeps vault keys out of the normal GUI's data flow, but it cannot guarantee that they are erased from memory. It also cannot protect them from an attacker able to inspect the broker's memory.

4.3 User Approval and Limited Permissions

The temporary permission token introduced in Chapter 1 is also called a capability. Figure 4.3 follows it through four stages: request, approval, issue and use. Before asking for approval, the broker checks the requested action and record. On Windows, the user first sees the broker's confirmation dialog and then Windows Hello. If approval fails or is cancelled, the broker does not issue a token for that action.

To create a token, the broker generates 32 random bytes and represents them as a URL-safe string. "URL-safe" describes how those bytes are written as text; it does not mean the token is sent to a website. The broker stores a SHA-256 digest of the token alongside the allowed record, action and expiry time. It can match a returned token to this stored entry without retaining the original string. The digest is only used to look up the permission. It does not encrypt data or unlock the vault key.

A token normally expires after 10 seconds. The capability manager permits a configured lifetime of up to 30 seconds and holds no more than 32 unused tokens at once. It measures time with a monotonic clock. This clock tracks elapsed time without being affected by changes to the displayed date and time. Expired entries are removed when the token store is checked or another token is issued. A token is rejected as soon as it reaches its expiry time.

At the token-use check, the broker first removes the matching stored entry. It then checks the record, action and expiry time. This means an attempt to use the token for the wrong record or action uses it up too. The caller cannot reuse that token to try other actions. Some invalid requests are rejected earlier and never reach this check. Requests on the broker connection are processed one at a time. When the session closes, all remaining tokens are cancelled.

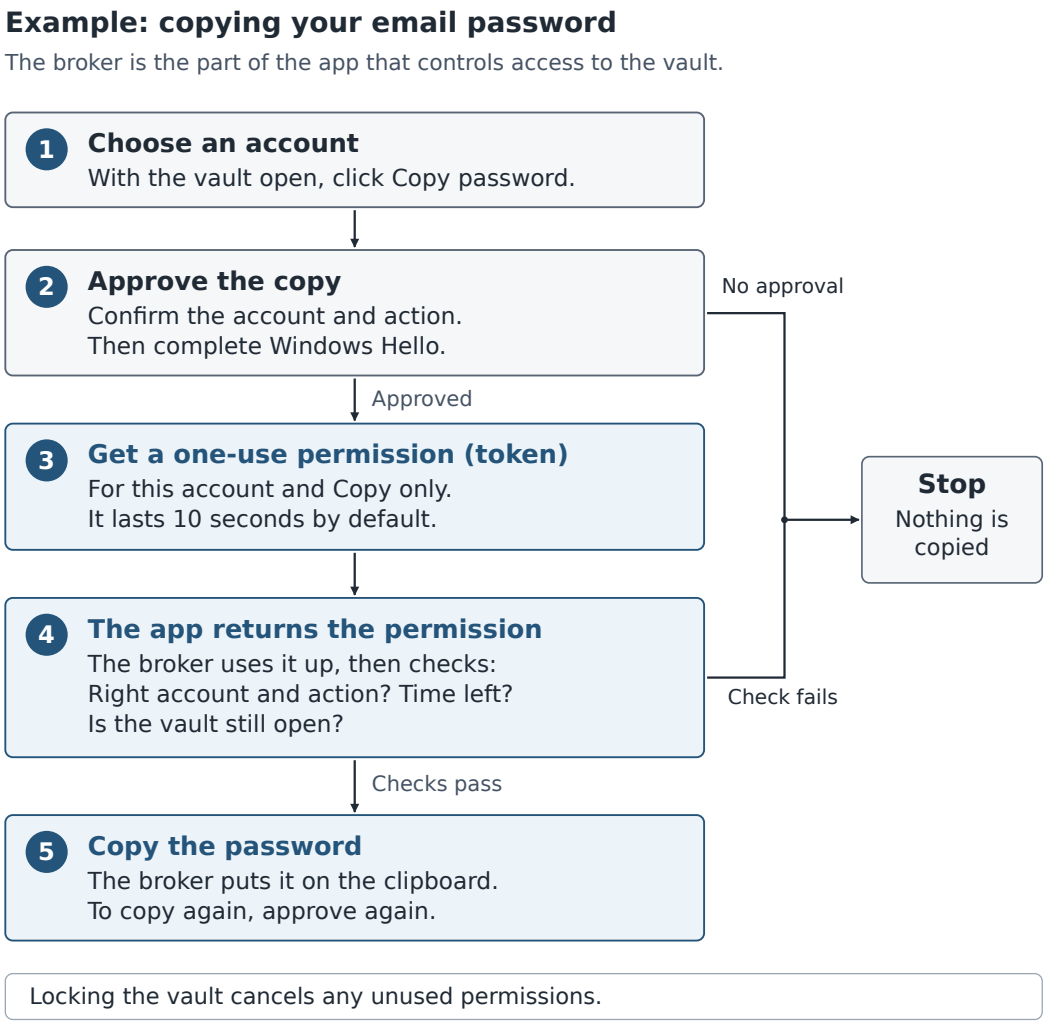


Figure 4.3: Approval and Single-Use Permission Flow

For example, a token approved for copying an email account's password cannot also allow that account to be deleted. Nor can it reveal the password of a shopping account. Once the copy attempt uses the token, copying again requires fresh approval. This example illustrates the permission rule; it does not describe the result of a malware experiment. Requiring another approval also makes repeated actions take more effort for the user.

The permission stored alongside the token's digest does not fix every detail of the operation. The stored entry includes the record identifier and action, but not the record version, password value, proposed edit data or backup destination. Some of these details are supplied or read only when the token is used. The confirmation therefore approves the account and action without fixing all the data involved. A change between approval and use can affect what happens. The Tamarin model makes a stronger simplification: its permission includes a fixed password. Section 6.3 explains this difference.

4.4 Credential Use and Locking

Copying and revealing a password work differently. After an approved copy request, the broker writes the password to the Windows clipboard and tells the GUI that the action succeeded. The password is not included in that response. After an approved reveal request, the broker sends the password to the GUI so that it can be displayed. Copying avoids sending the password in the response, but the GUI or another program may still read it from the clipboard. The password can therefore still be exposed when it is copied.

After 30 seconds, the application tries to clear the copied password. It first checks that the clipboard still contains the value it placed there. If the user has since copied something else, the newer text is left alone. Revealed passwords are normally displayed for 20 seconds. These limits shorten the time passwords remain available during normal use. They cannot erase a copy already captured by another program or remove it from a GUI controlled by an attacker.

By default, the application locks after five minutes of inactivity. The user can change this period in Settings. There is also an emergency lock shortcut while the application is active, along with handling for related session and power events. Closing the broker session cancels unused tokens and prevents further normal vault operations. To continue, the user has to unlock the vault again.

Some events that trigger locking, including inactivity detection, are handled first by the GUI. This creates a limit under the chosen attacker model: an attacker controlling the GUI could stop it from asking the broker to lock. The broker closes the session when it receives the request, but the GUI timer cannot independently protect against the GUI itself. One possible improvement would be for the broker to enforce the idle deadline directly.

4.5 Backups and Audit Records

A backup is a copy of the encrypted vault. Creating it requires broker approval and a new destination file, which avoids overwriting an existing file. The backup still needs the

master passphrase for the state it contains. If the working vault is lost or damaged, this copy can help, provided it remains available and can be opened.

The copy cannot recover a forgotten passphrase. It may also contain older records, and restoring it brings back that earlier state. The application warns about some folders that synchronise with cloud services. It cannot detect every way a file might be uploaded, so the user remains responsible for choosing where to keep backups.

Audit records keep a local history of selected events, including failed and denied actions. The allowed fields exclude passwords, passphrases and tokens. Each entry is linked using keyed hashes, which are checks calculated with a secret key. These links can reveal some changes to the log, provided the key and the expected records remain trustworthy.

The audit key is saved locally under the same user account as the log. An attacker who obtains both may be able to rewrite them. Linking the entries cannot prevent all the local evidence from being deleted. A failure to write the log also does not stop every application operation. The log can therefore support an investigation, but cannot provide a complete guarantee about what happened.

4.6 Design Decisions and Trade-offs

Keeping request checks in the broker makes them easier to apply consistently. However, approval for each sensitive action adds work for the user. The user also remains responsible for backups, and both processes depend on the same computer.

Listing and integrity checks decrypt records inside the broker, and searching uses the same listing path. Although the GUI receives only summaries, many passwords may be readable in broker memory. Reducing this internal decryption would require changes to how the records are stored and searched.

Chapter 5

Implementation

5.1 Project Structure and Tools

The prototype is written in Python, with its code in the `password_manager` package under the `src` folder. The final project version is 0.8.0, and the project declares support for Python 3.12, 3.13 and 3.14. PySide6 supplies the windows and dialogs. The `argon2-cffi` library handles the Argon2 calculation, and `cryptography` supplies AES-GCM and HKDF. The vault's structured data is stored with SQLite. Tests, installation instructions and a script for converting an older vault format are also included.

The code is divided into the areas shown in Table 5.1. For instance, the broker code checks permissions, while the interface code controls how an account row looks. Giving each area a clear job makes it easier to find where a change belongs and to test smaller parts separately. Architecture tests check selected rules that keep direct access to the vault out of the normal GUI.

Table 5.1: Main Implementation Areas

Area	Responsibility
`core`	Account data, input checks, passphrase handling, password generation and encryption helpers.
`storage`	Vault files, encrypted records, checks for changes, updates and backups.
`broker`	Starting the separate process, handling messages, checking approval and carrying out allowed actions.
`security`	Activity tracking, audit records, file checks and related Windows events.
`ui`	Main window, dialogs, user actions and display behaviour.
`tests`	Automated checks of individual parts, requests and selected interface behaviour.

The application is installed from its source code. A Python virtual environment keeps the packages it needs together, and the lock file records which packages and versions to install. Development tools such as Ruff and mypy are listed separately. They check the code and are not part of managing passwords. Appendix A gives the installation commands from the project README.

5.2 Encrypted Vault and Credential Operations

The storage code uses a vault session to represent an unlocked vault. To create a vault, the application first checks the passphrase and destination file. It then generates the random keys and other values described in Chapter 4 and writes the encrypted vault. To open an existing vault, it checks the file format and saved encryption settings before using the passphrase to try to unlock the vault key. If the passphrase is wrong or a data check fails, no open session is created.

A new master passphrase must contain at least 15 characters and pass the application's other local input checks. This report does not list every local check separately. Spaces and uppercase or lowercase letters are kept as entered. The program also applies Unicode NFC normalisation, which gives certain equivalent character sequences the same internal representation. This step does not remove spaces or change uppercase letters to lowercase. The same process is used when unlocking the vault. These checks help reject unsuitable input, but they do not measure how difficult every accepted passphrase would be to guess.

Before saving an account, the program checks its fields. It accepts the defined text fields and rejects values of the wrong type or an excessive size. JSON represents the account data in a structured format, while record identifiers and versions link that data to its encryption checks. The storage code handles changes to a record together with the matching manifest update. Individual windows therefore do not need to repeat the encryption steps. SQLite transactions help keep these related changes together.

The code sets four size limits: 10,000 records per vault, 64 KiB for one record's data, 2 MiB for the manifest and 512 MiB for the vault file. These are limits on accepted input, not a record of the sizes tested. The report does not establish that every size up to the maximum has been tested. There are also no performance measurements showing how a vault at the maximum size would behave on different machines.

Building the account list involves more than reading the summary fields. The `list_credentials` function reads and decrypts records to create account objects, and searching uses the same list. The broker then selects only the summary fields for its response to the GUI. Passwords are left out of the response after they have been decrypted inside the broker. This limits what is returned, but does not keep every password encrypted while the list is built. A separate search index could be considered in future work to reduce this decryption.

An account can be edited without displaying its old password. The client can ask to keep the saved password unchanged, and the broker can retain it without sending it back to fill the editing form. A new password typed or generated in the GUI is already readable there before it is sent for storage. The broker cannot hide a password from the process in which it was entered. The design mainly controls access to records that are already saved in the vault.

5.3 Broker Communication and Permission Checks

The application starts the broker as a child process: a separate process launched by the main application. The vault creation and unlock dialogs collect input through the broker's

path. The normal main window neither derives nor holds the vault key. A separate random authentication key is used when establishing the local connection between the processes. Its job is to check that connection. It is not used to encrypt the saved vault.

On Windows, the processes exchange messages through a local named pipe, which is a Windows communication mechanism. The connection code also checks that the process at the other end belongs to the current user. Each message contains JSON sent as a byte string with a size limit. The application does not send request objects that require Python's pickle loading. This gives it a defined message format that it can check before carrying out an action.

A valid connection does not make every request trustworthy. The chosen threat model assumes that the attacker already controls the authenticated GUI client, so unwanted requests can arrive over that connection. Each operation still needs to be checked by the broker. The application does not add another encryption layer to each JSON message. Checking who can connect and encrypting the vault are separate jobs.

A request contains an identifier, the operation name and the data that operation needs. The protocol checks the message structure and its allowed values. The service then checks the fields for the particular action and rejects unsupported operation names. Each page of a list response can contain up to 50 summaries, and a search query can contain up to 200 characters. These limits control the size of responses and queries. However, the storage code still builds the full underlying list before the broker selects a page to return.

The operations allow the client to list or search summaries, request or use tokens, lock the vault and check its status. Creating a record or a backup uses a designated identifier. These actions do not refer to an existing account in the same way that revealing or deleting a record does. Other sensitive requests must identify an existing record before approval is requested. If an approval policy has not been supplied, the default policy denies sensitive access.

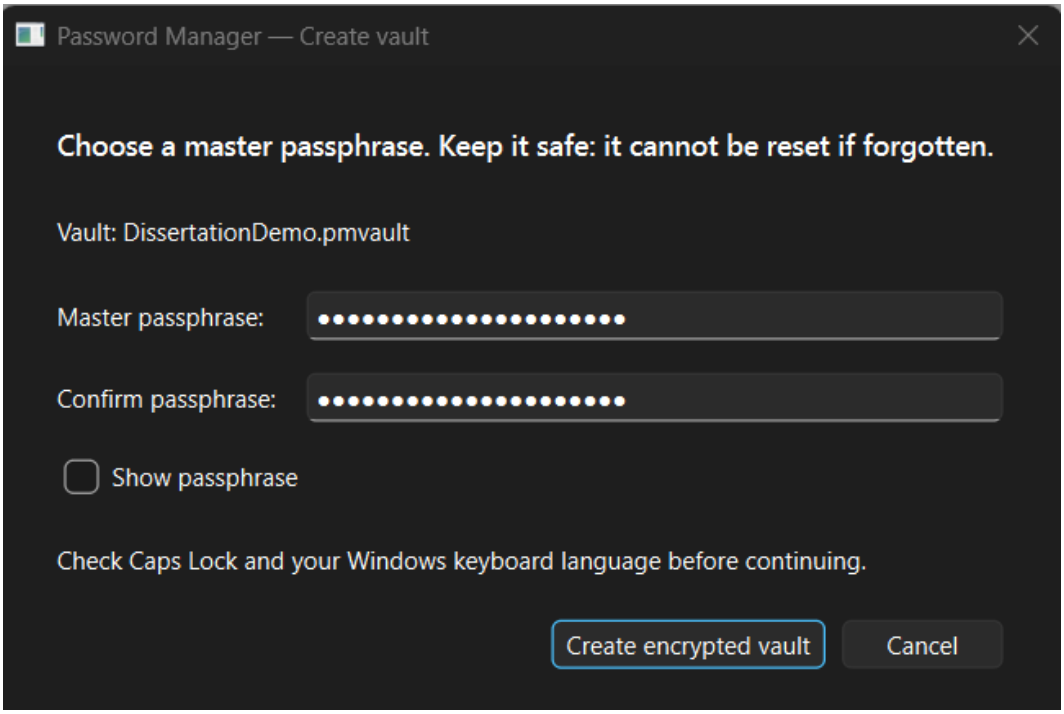
When the client uses a token, the service checks its permitted record and action before calling the required vault operation. The response depends on the action. View returns the record details without its password. Reveal returns the selected password. Copy places the password on the clipboard and returns a value indicating that it was copied. An invalid request should not lead to another action with fewer restrictions. If an unexpected error occurs while handling a request, the broker session closes so that operations do not continue with an uncertain session state.

5.4 User Interface and Windows Hello

The opening screens let the user create a new vault or select and unlock an existing one. Figure 5.1 shows these dialogs with demonstration data.

After the vault opens, the main window displays account summaries, search controls and record actions, as shown in Figure 5.2. The user chooses an account before requesting an action. They can therefore see which account they are working with, while its password stays out of the list.

(a) Vault creation



(b) Vault unlock

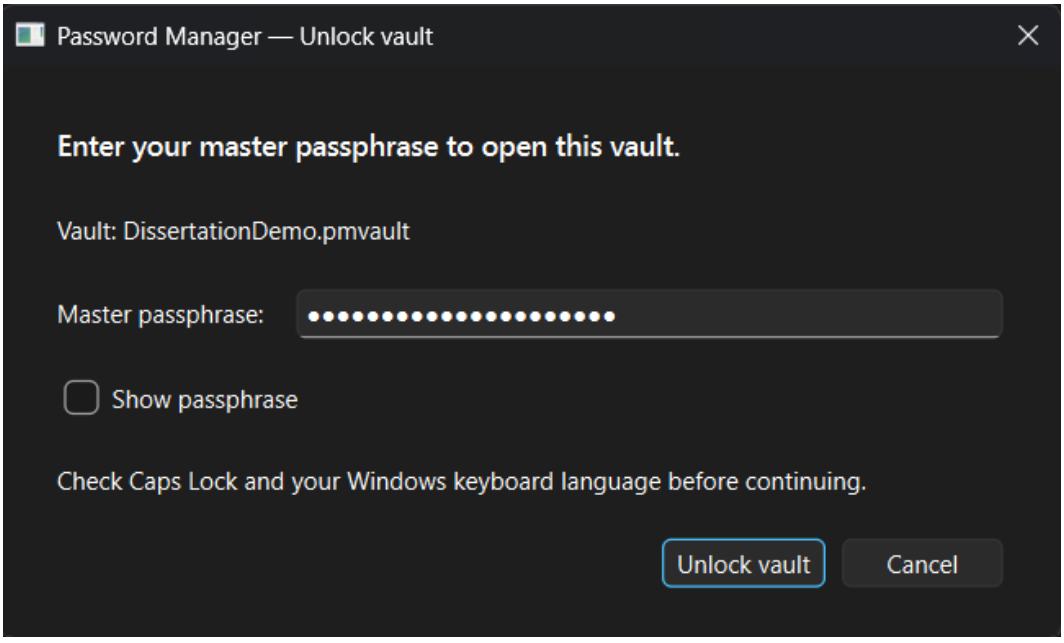


Figure 5.1: Vault Creation and Unlock Screens

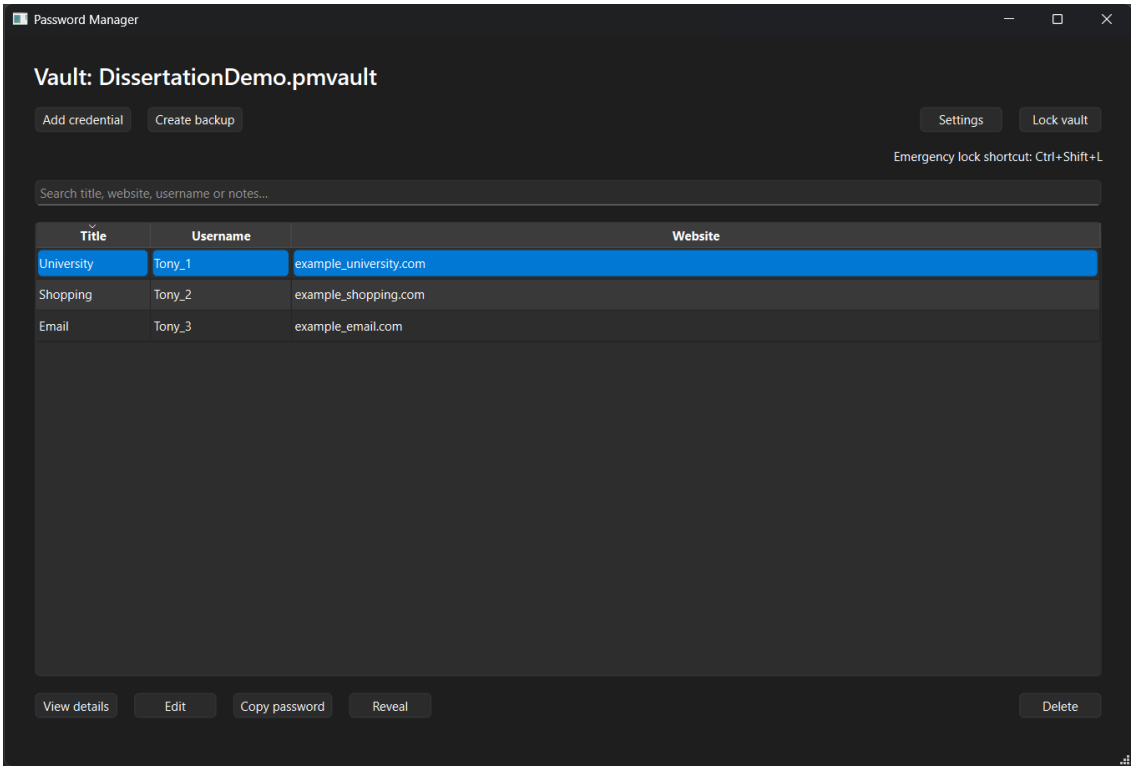


Figure 5.2: Main Credential List

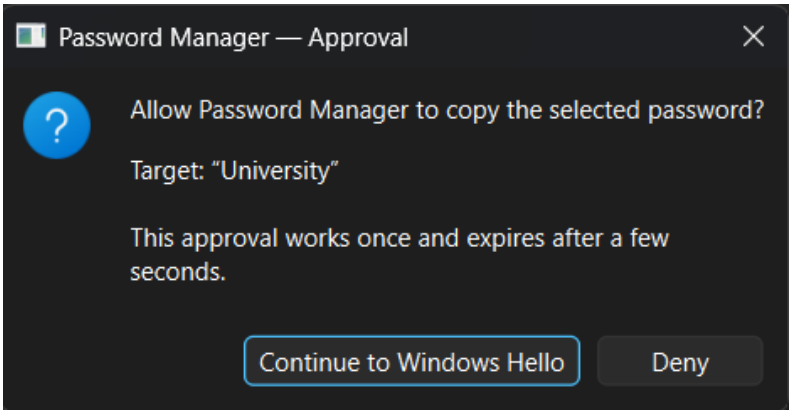
Figure 5.3 shows the broker confirmation and Windows Hello approval steps for sensitive actions. The application requests verification through Windows `UserConsentVerifier`, an operating-system interface. Windows uses the method configured for the account, such as a PIN, fingerprint or face verification when available [21]. The password manager does not compare fingerprints or recognise faces itself.

The broker confirmation names the action and, for an existing record, its account title. A successful Windows check lets approval continue. The action is denied if verification fails, is cancelled or is unavailable. The broker confirmation is an ordinary dialog on the normal desktop, not a protected Windows secure-desktop prompt. A successful check cannot show whether the user understood the action's consequences. Some automated tests use substitute verification results to check these outcomes.

Figure 5.4 shows the password generator and account-editing form. The generator uses Python's `secrets` functions for random choices. Its default length is 20 characters, with lengths from 12 to 128 accepted. Users can choose lowercase letters, uppercase letters, digits and symbols. It takes at least one character from each enabled group, fills the remaining positions from the combined groups and shuffles them using secure random choices. Invalid lengths and choices with no enabled groups are rejected.

Websites may set their own password rules. The application does not contact websites to check whether they accept a generated password or to change account passwords. The user must set the password at the service and save the same value in the vault. This manual step remains part of the workflow. Browser integration could change it, but is not implemented.

(a) Approve the action



(b) Verify with Windows Hello

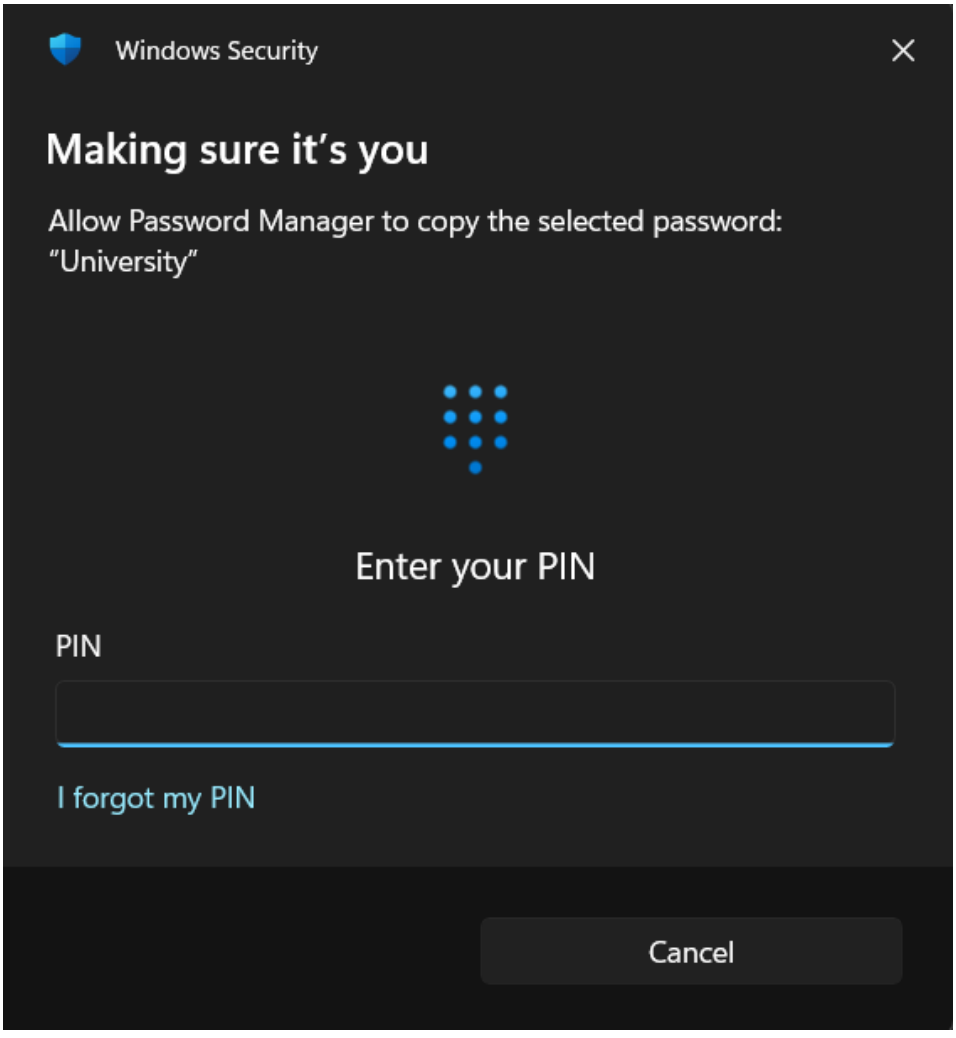
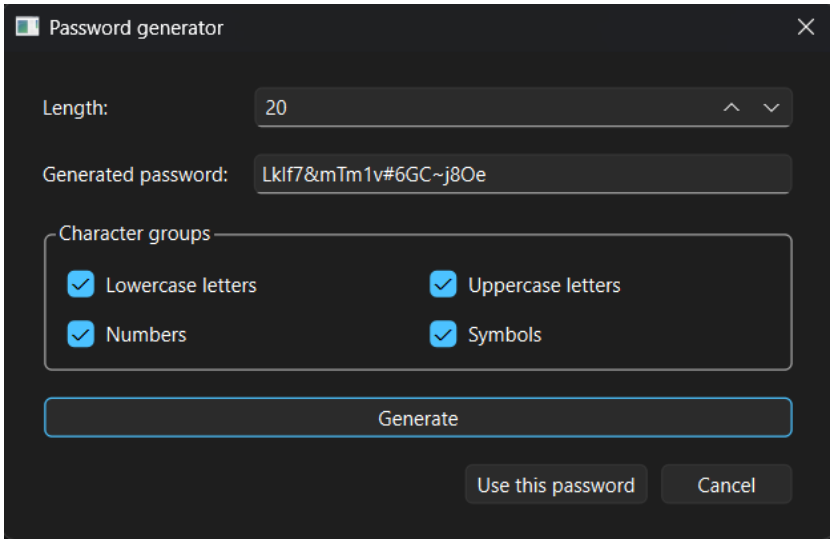


Figure 5.3: Sensitive Action Approval and Windows Hello

(a) Generate a password



(b) Edit the account details

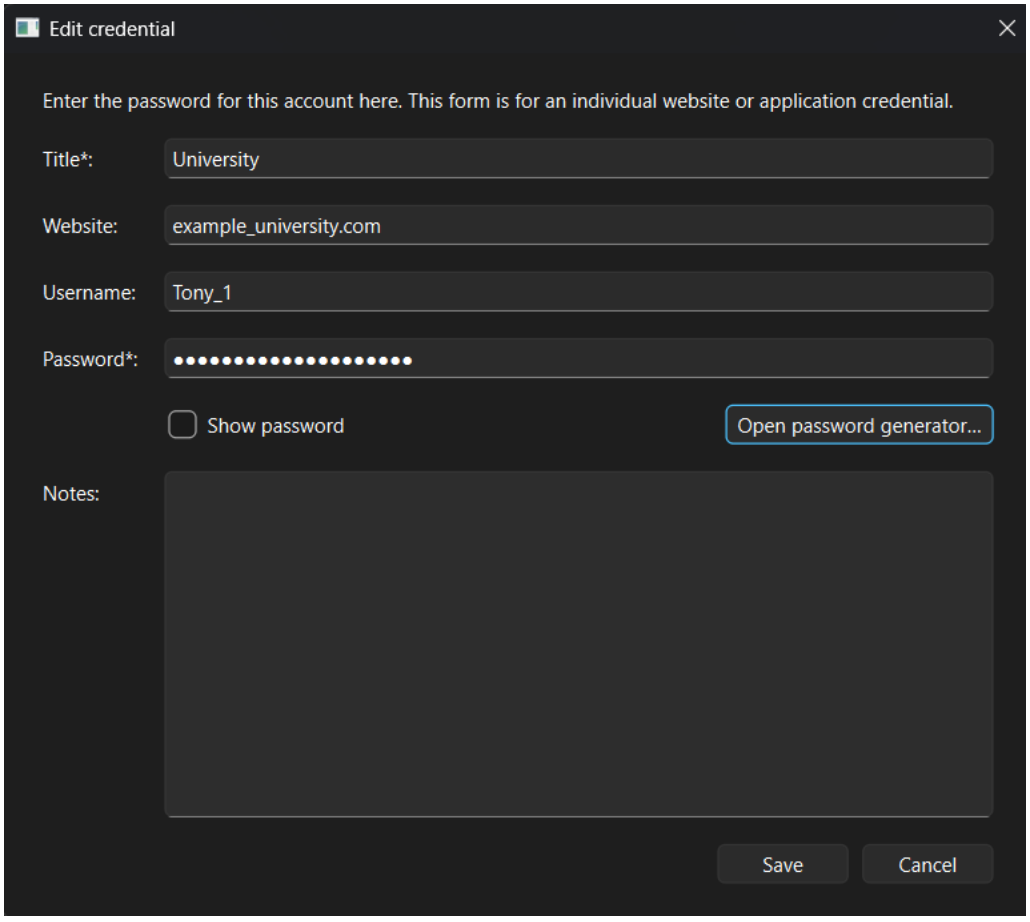


Figure 5.4: Password Generation and Credential Editing

5.5 Clipboard, Locking and Audit Logs

The broker performs the sensitive step of writing to the clipboard. It retrieves the selected password and gives it to the Windows clipboard helper. A temporary clipboard guard keeps a digest so that it can recognise the copied value later. When the timeout is reached, the guard checks the clipboard and clears it only if that value is still there. Different text copied afterwards is left alone. Targeted tests check this cleanup during normal use.

The GUI monitors activity and checks the inactivity period using a Qt timer. Once that period is reached, the controller requests a lock. Related handling also responds to Windows session and power events. While the application is active, the user can lock it manually with the emergency shortcut. Locking clears the account display and any temporary password reveal. The broker session is then closed and its tokens cancelled. Simply hiding the window would not do these things.

The audit helpers accept a selected set of event fields and link log entries with keyed hashes. This allows some failures and denied requests to be reviewed without writing passwords into a diagnostic file. Saving complete requests in the log would be unsuitable because they could contain account details or tokens. The code also handles errors that occur while writing the log. Not every operation requires a successful log write to continue, so this history does not provide a complete record of all activity.

Together, these parts support everyday account tasks and apply the request checks. The next chapter examines the recorded results, including which behaviours were tested and why some tests were skipped. It also explains where the Tamarin model simplifies the application's real behaviour.

Chapter 6

Testing and Evaluation

6.1 Evaluation Approach

The evaluation asks two main questions: can the prototype carry out the required password-management tasks, and does the broker enforce the chosen rules when the GUI sends a request? This chapter answers them using the automated test report, code-quality and dependency logs, and Tamarin proof outputs. Reading the source code helps explain what these results mean for the implementation. A pass result alone does not show how a feature works or which assumptions it depends on.

The automated results come from `docs/test-results.xml`, which is included with the final application source. This file records 312 test cases run on 14 September 2026, taking 21.267 seconds in total. That duration applies to the test run. It does not measure how quickly the application unlocks or responds during use. Appendices B and C list the evidence files. The analysis uses these recorded results; it does not include any additional test or proof runs.

The requirements in Chapter 3 show which parts of the project these checks address. Tests of vault creation and account operations cover parts of FR1–FR3 and SR1. Broker and token tests address FR4 and SR2–SR4. Lock and clipboard tests cover parts of FR5 and SR5. Backup tests relate to FR6, while input and error checks relate to FR7. Audit tests address SR6. The interface and user guide show how the usability requirements were approached, but there are no measurements from participants for UR1.

Each method answers a different kind of question. An automated test can supply a controlled approval result and check whether the requested action follows. It cannot show how often a person would accept a misleading prompt. Tamarin checks the rules of a simplified model; it does not run the complete Windows application. These differences set the limits of what the results can show.

6.2 Functional and Security Test Results

The recorded run contains 305 passed cases and seven skipped cases, with no failures or errors. Table 6.1 summarises the main results. The test report covers 28 modules.

Some test functions use several inputs, so one function can produce multiple test cases. Appendix B lists each module's results and the reasons for skipping tests.

Table 6.1: Summary of Recorded Implementation Checks

Check	Recorded outcome
Automated tests	305 passed; 7 skipped; 0 failures; 0 errors.
Ruff	All checks passed for the selected rules and paths.
mypy	No issues found in 39 source files in the configured scope.
Dependency consistency	`pip check` reported no broken requirements.

The storage and encryption tests cover ordinary use as well as attempts to open invalid data. They check saving and reopening records, entering a wrong passphrase, changing encrypted data and using invalid encryption settings. Manifest tests also check changes to the saved collection, beyond changes within an individual record. These results do not establish protection against complete-file rollback. An old valid vault may still pass the application's checks if it is restored together with its matching manifest.

Tests of account operations cover creating, editing, deleting and searching records. The broker tests check that account summaries contain neither passwords nor notes and that searches return matching summaries. Another test checks that the supported interface has no bulk operations. A caller therefore has no normal request that returns all passwords. This does not mean that records stay encrypted throughout the process: the broker decrypts them internally before returning the summaries.

The token tests give the clearest evidence for the request rules. They check that each token can be consumed only once, and that a wrong record, wrong action or expired token is rejected. They also check invalid lifetime settings, the limit on unused tokens and token cancellation when the vault locks. Service tests go further by sending altered actions, tokens for other records and repeated requests through the request path. These tests examine how the broker service uses the token manager, as well as checking the token manager itself.

The copy tests check where a password goes after approval. The broker uses its configured clipboard writer and returns a status saying that copying is complete. Its response does not contain the password. This supports the stated response format, but it cannot show that the GUI or another program is unable to read the clipboard afterwards. The clipboard-clearing tests check the intended timeout and whether the contents are checked before clearing. Clearing cannot remove a password that has already been captured elsewhere.

Approval tests provide controlled outcomes for both accepted and rejected requests. Sensitive access is denied if no approval policy has been supplied. Rejection or cancellation prevents the broker from issuing a token. Further tests cover invalid messages, unsupported operations and unexpected failures. An unexpected failure in a request handler closes the session. Some Windows Hello checks replace parts of the verification process with controlled substitutes. They therefore do not provide a full record of real PIN and biometric checks across different devices.

The backup tests check that approval is required and that the copied vault remains encrypted. Storage and conversion tests cover other selected file operations. Audit tests check the permitted event fields and the behaviour of log verification. This gives evidence for the features tested, but does not cover every disk failure, recovery situation or possible change to a log.

The seven skipped cases are not passes. Six check Linux-specific transport behaviour, including runtime directories, socket paths and peer checks. The seventh is a Windows test that checks whether a symbolic link is rejected. A symbolic link is a file reference that points to another file. The account lacked permission to create the link needed for the test, so the test could not run. This run therefore provides no successful result for those seven cases.

Ruff, mypy and `pip check` examine other aspects of the project. Ruff checks selected code rules in the source, tests and scripts. mypy checks types in 39 source files using the project's settings. `pip check` checks whether the installed packages have consistent requirements. A clean result from this command is not a check for known security vulnerabilities. These tools do not prove that the entire application is secure. No coverage percentage is reported because the supplied evidence contains no completed code-coverage report.

6.3 Tamarin Model Results

Tamarin works with a simplified model made from symbols and rules. In this project, the rules describe request approval, token use, expiry and locking. A sequence of events in the model is called a *trace*. A *lemma* is a statement that Tamarin is asked to check. An exists-trace lemma asks whether at least one intended sequence can happen. An all-traces lemma asks whether a stated property holds for all sequences allowed by the model [22].

Both kinds of check help explain the results. For example, a model that blocks every password release might satisfy some safety rules simply because no useful action can occur. The executability check confirms that an approved reveal is possible. The remaining lemmas check the restrictions on release, including the need for earlier approval and the rule against using a token twice.

The executability result has a limited purpose: it shows that the model allows an intended approved release. It does not, by itself, establish that every relevant situation is represented or that every safety property is non-vacuous. A property is vacuous if it holds only because the situation it concerns cannot occur. The executability comment in the model listing in Appendix C should therefore be understood as ruling out a model that blocks every release, rather than establishing meaningful coverage for every safety property.

The final model starts with a new vault key, record, password and identifier called a generation. This identifier separates each new model instance from the others. An attacker can send requests. A trusted approval rule then creates a token for the record, action and generation. The model stores this permission as a state fact that can be consumed once. Using the token, expiring it or locking the relevant state removes the fact needed for a

later release. The attacker can observe and resend public messages, but cannot change the broker's private state.

Six separate proof runs are recorded in Table 6.2. Each run checked one selected lemma. The other lemmas appear as incomplete in that run, but each has a separate output confirming its own verification. The detailed logs match the combined summary. The version record lists Tamarin 1.12.0 and Maude 3.5.1. Further build details are included in Appendix C.

Table 6.2: Recorded Tamarin Results

Property	Type	Result	Steps
Intended execution is possible	Exists-trace	Verified	7
Approval precedes release	All-traces	Verified	3
Release matches issued scope and password	All-traces	Verified	3
Expiry and lock prevent late release	All-traces	Verified	18
A capability cannot be replayed	All-traces	Verified	8
The vault key remains secret	All-traces	Verified	2

The first result shows that setup, request, approval, token issue and reveal can take place in the intended order. The approval result checks that each release follows an earlier approval for that record and action. Together, the record and action define the token's scope. The matching result connects a release to its issued token and to the password fixed in that model instance. The expiry and lock result prevents a release after the relevant expiry or lock. The replay result checks that one token cannot cause two separate releases.

There are important differences between this model and the Python application. Each fresh generation follows one record and one token lifecycle. Setup can repeat to create independent instances, but this does not represent many records and up to 32 unused tokens within one real vault session. Record edits are also left out. In the application, a token checks the record and action without fixing the password or record version. The model's password-matching result therefore does not prove that the program checks record versions.

The model also simplifies time and approval. Expiry happens as an event, rather than as a measurement of the application's ten-second clock. The lock rules assume that a lock event occurs; they cannot establish that a compromised GUI will send one. Approval is trusted, without modelling Windows Hello, the appearance of the prompt or what the user understands. The release rule outputs a password. It does not reproduce each real action separately, including record editing and encrypted backup creation.

The vault-key result must be read within these same boundaries. The model creates the key privately and never outputs it. It leaves out the Argon2id, AES-GCM and HKDF calculations, Windows process permissions and copies of data in memory. The result checks the private-key rule in the model. It does not verify those parts of the application. There is no machine-checked proof that the complete Python code follows the model. The comparison here is based on reading their rules, behaviour and assumptions.

6.4 Evaluation Limitations

The evidence covers the selected program tests and the rules expressed in the model. It does not measure a reduction in password theft during everyday use. There is no controlled comparison with an application that performs all vault operations in its main process. No attack test gives the GUI unrestricted code execution, and there is no systematic measurement of how long passwords stay in memory. The results cannot therefore support a numerical claim about reduced exposure.

The recorded test environment also limits what can be concluded. Seven cases were skipped, and controlled results replaced some Windows behaviour. Windows Hello cancellation, permissions, clipboard interaction and session events need further checks on real hardware. The duration of the automated test run does not measure application performance or behaviour with large vaults.

No participant study measured task times, satisfaction or whether users noticed unexpected approval requests. Informal feedback that the application works is helpful during development, but it is not a measured study. Repeated prompts deserve particular attention because the design depends on users approving sensitive actions.

The tested rules limit a token to its record, action, lifetime and use count. Tamarin checks related rules in a smaller model. Chapter 7 considers the additional isolation, modelling and practical testing needed before making wider claims.

Table 6.3 brings together the main findings and their limits. It connects the research question to the checks described in this chapter; it does not add new evaluation results.

Table 6.3: Main Claims, Existing Evidence and Limits

Claim	Existing evidence	What remains outside the finding
Account summaries exclude passwords and notes.	Broker tests check list and search responses.	The broker still decrypts records internally. These checks do not measure readable data in memory.
Tokens restrict the record, action, lifetime and number of uses.	Token and service tests check wrong records, changed actions, expiry and repeated requests.	Tokens do not bind the record version or every action detail. A password released after approval may still be retained.
A copy response does not contain the password.	Copy tests check use of the clipboard writer and the returned status.	The password is placed on the clipboard, where another program may read it.
Approval precedes release in the model.	The recorded all-traces lemma is verified.	Approval is trusted in the model. The result does not check user understanding or the complete Windows approval process.

Chapter 7

Discussion and Conclusion

7.1 Answer to the Research Question

The research question asks how much a separate vault broker can limit the credentials that a compromised GUI obtains through normal requests. This project shows that the broker can restrict that access, provided the broker and operating system remain trusted. During ordinary browsing, the interface returns account summaries. It has no operation that releases all passwords at once. A sensitive action needs approval, followed by a token that allows one action on one record. That token expires after a short time and can be used only once.

Table 6.3 brings together the source-code findings and recorded tests that support these rules in the cases examined. A token approved for one password does not allow access to the whole vault, and a consumed token cannot be used successfully again. Tamarin checks related rules within its model, including approval before release and prevention of a token's second use. In the application, the broker carries out the permission checks. The rules therefore apply to requests sent to the broker, rather than depending only on the main window's buttons and controls.

Approval does not stop a password from being captured once it is released. Revealing sends the password to the GUI; copying puts it on the clipboard. An attacker able to read either destination may obtain it. More approvals can allow more passwords to be released. The prototype checks that the requested record and action match the approval, but it cannot judge whether the request makes sense as part of the user's wider task.

This answer also depends on what the attacker can reach. The threat model excludes direct access to read or change the broker, even though both processes run on the same computer. The evidence therefore supports a limited conclusion about access through the normal broker interface under these assumptions. It does not establish protection against all malware or give a percentage reduction in stolen passwords.

7.2 Comparison with Related Work

Tapas is the closest comparison because it also seeks to limit password exposure while passwords are being used [3]. Here, a local broker handles that role; Tapas uses a second

device. The prototype does not need a smartphone, but its two processes depend on the same computer and Windows permissions. It cannot claim the separation that Tapas gains from two devices, and Tapas's user-study findings do not show how people would use this application.

The web password-manager and autofill studies show why the details of a request matter. Li and colleagues, Oesch and Ruoti, Leaky Autofill and AutoFail describe ways in which request handling or missing context can expose passwords [2, 12–14]. This project applies that lesson by checking the account and action in the broker. It does not repeat those studies' experiments. The current token checks also leave a gap: they do not fix the record version or all the data involved in an action. Checking more of those details remains necessary future work.

HIPPO and the cloud-manager analysis consider different places to hold information and different parts of the system that must be trusted [4, 7]. HIPPO generates passwords through a protocol, while cloud managers use a remote service. The prototype keeps credentials in an encrypted local vault. These results do not provide a fair basis for ranking the approaches. Their suitability also depends on recovery needs, risks to the devices and everyday use. The contribution here is a working implementation with evidence about its own design.

7.3 Project Limitations and Lessons Learned

One lesson from the project is that each security feature needs a clear explanation of what it does. Calling a component a broker does not remove its dependence on Windows process permissions. Windows Hello approves sensitive actions, while the master passphrase is still needed to unlock the vault. An encrypted backup protects the saved copy but cannot recover a forgotten passphrase. Tamarin also has a specific role: it checks the model's rules, not every part of the application.

Looking at where passwords become readable helps explain the limits of the design. The main window does not normally receive all passwords, but the broker decrypts records to list them and check their integrity. A copy response leaves out the password, yet the password is placed on the shared clipboard. Closing a session removes active key references without proving that every copy has been erased from memory. Leaving passwords out of the GUI's list therefore does not mean that only one password is decrypted inside the broker.

The 305 passed tests need to be read alongside their inputs and the seven skipped cases. Likewise, passing the selected code checks does not certify the application's security. The comparison with Tamarin shows another limit: each model instance follows one permission lifecycle with a fixed password, whereas the application allows records to change and can hold several unused tokens. The proofs do not establish how every combination of these actions behaves in the application. Those differences identify where further checks would be useful.

There is also a gap between having a working interface and knowing whether people find it easy to use. The available evidence contains no participant study or application benchmark. The report therefore cannot give firm results about approval fatigue, response

times for large vaults or advantages over commercial managers. Users may need to approve actions repeatedly during normal tasks, and that experience still needs to be studied. The strongest evidence concerns the implemented request controls and the specific properties checked in the model.

7.4 Future Work

Protecting the broker more strongly should be the first priority. This would involve identifying the Windows process-access permissions it needs and testing them against the intended attacker. Further work could also examine a more isolated environment for the broker and a way for users to recognise a prompt from the trusted component. Both ideas need practical testing. Simply starting another ordinary process would leave the same dependence on one computer and user account.

The approval should also cover more of the requested action. A token could include the record version and a digest, or fingerprint, of the operation's data. The prompt would need to show the same details that the broker checks when the token is used. If the password record, proposed edit or backup destination changes, a new matching approval would then be required. Tests should try changing these values between approval and use, as well as using several unused tokens in the same session.

Giving the broker its own inactivity deadline would reduce its reliance on the GUI to request a lock. Before doing this, the design would need to define which messages count as activity. Otherwise, an attacker could keep sending requests to prevent the session from closing. Tests should include missing, delayed and repeated messages, with the expected locking time stated clearly.

Practical evaluation could address several remaining questions. A controlled comparison could send requests to the broker design and to a version without the same separation. Real Windows hardware tests could record how Windows Hello, the clipboard and session events behave. Measuring unlock, search and backup times for different vault sizes would help assess performance. Memory inspection under defined conditions could show where passwords remain readable. Together, these checks would support more precise conclusions about the application's behaviour in practice.

A participant study could then examine whether people understand the approval prompts and notice unexpected requests. It could measure task completion and how repeated approvals affect their use of the application. The required approval must be obtained before any participant data is collected. This would help establish whether the extra security steps are manageable in everyday use.

The formal model could be extended to cover several records and tokens in one session, record changes, and separate copy, reveal, edit and backup actions. Connecting these rules more closely to implementation tests would make the comparison easier to assess. Proving that the complete program follows the model would be a much larger task. Synchronisation, passkeys and honey vaults are possible additions, but each would need its own design and evaluation before being included [5, 18].

7.5 Conclusion

This project examined access to saved passwords after a vault is unlocked. The result is a working local Windows password manager with a separate vault broker. Users browse account summaries in the main window, while the broker performs vault operations and checks sensitive requests. Keeping those responsibilities in the broker gives the application one clear place to enforce its rules for credential use.

The prototype combines encrypted storage with approval for individual actions. Argon2id processes the master passphrase, and AES-256-GCM protects stored data and checks for changes. Windows Hello is part of the approval process. After approval, a temporary token allows one action on one record and only one use. Password generation, encrypted backups, clipboard clearing and locking provide the other functions needed for everyday password-management tasks.

The recorded automated run contains 305 passed tests and seven skipped tests, with no failures or errors. The code and package checks also passed with the project's settings. In Tamarin, one intended execution was shown to be possible and five properties were verified in the simplified permission model. These results support the selected request rules examined in the report. They do not prove that every part of the Windows application is secure.

The work addresses the objectives in Chapter 1. The literature review compares related designs and explains what they need to trust. The prototype separates encrypted vault operations from the main window and checks approval, token scope, expiry and reuse. Recorded automated checks cover selected features and security rules. Tamarin examines a smaller model, with the differences from the application explained in the analysis. The discussion sets out what the findings support and what still needs investigation. Meeting these objectives means building and examining the prototype; it does not demonstrate measured gains in everyday security, performance or usability.

The supported answer to the research question is that the broker can limit what a GUI obtains through normal requests while the broker and operating system remain trusted. A copied or revealed password can still be captured, and records may be readable in broker memory. Tokens do not fix every detail of an approved action either. These limits explain why the design cannot offer complete protection against malware.

Further work should strengthen broker isolation, check approved actions more fully when tokens are used, and extend evaluation on real hardware and with users. A closer formal model could examine record changes and several tokens within one session. The prototype provides a working example of controlled password release, with recorded checks and clearly stated limits. It gives further work on protecting credentials during use a practical starting point.

Bibliography

[1] J. A. Halderman, B. Waters, and E. W. Felten, "A convenient method for securely managing passwords," in *Proceedings of the 14th International Conference on World Wide Web*, pp. 471–479, 2005. doi: 10.1145/1060745.1060815.

[2] S. Oesch and S. Ruoti, "That was then, this is now: A security evaluation of password generation, storage, and autofill in browser-based password managers," in *29th USENIX Security Symposium*, pp. 2165–2182, USENIX Association, 2020. `https://www.usenix.org/conference/usenixsecurity20/presentation/oesch`.

[3] D. McCarney, D. Barrera, J. Clark, S. Chiasson, and P. C. van Oorschot, "Tapas: Design, implementation, and usability evaluation of a password manager," in *Proceedings of the 28th Annual Computer Security Applications Conference*, pp. 89–98, ACM, 2012. doi: 10.1145/2420950.2420964.

[4] M. Scarlata, G. Torrisi, M. Backendal, and K. G. Paterson, "Zero knowledge (about) encryption: A comparative security analysis of three cloud-based password managers," in *35th USENIX Security Symposium*, pp. 4267–4286, USENIX Association, 2026. Three-provider version. `https://www.usenix.org/conference/usenixsecurity26/presentation/scarlata`.

[5] H. Cheng, F. Huang, J. Yang, W. Li, and P. Wang, "Practically secure honey password vaults: New design and new evaluation against online guessing," in *34th USENIX Security Symposium*, pp. 7781–7798, USENIX Association, 2025. `https://www.usenix.org/conference/usenixsecurity25/presentation/cheng-haibo`.

[6] M. Jubur, P. Shrestha, and N. Saxena, "An in-depth analysis of password managers and two-factor authentication tools," *ACM Computing Surveys*, vol. 57, no. 5, 2025. Article 128, 32 pages. doi: 10.1145/3711117.

[7] M. Jubur, C. R. Price, M. Shirvanian, N. Saxena, S. Jarecki, and H. Krawczyk, "Building and testing a hidden-password online password manager," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 7454–7468, 2025. doi: 10.1109/TIFS.2025.3583459.

[8] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 memory-hard function for password hashing and proof-of-work applications," Tech. Rep. RFC 9106, IRTF, September 2021. `https://www.rfc-editor.org/rfc/rfc9106.html`.

[9] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC," Tech. Rep. SP 800-38D, National Institute of Standards and Technology, 2007. doi: 10.6028/NIST.SP.800-38D.

[10] M. Bellare and L. Shea, "Flexible password-based encryption: Securing cloud storage and provably resisting partitioning-oracle attacks," in *Topics in Cryptology – CT-RSA 2023*, vol. 13871 of *Lecture Notes in Computer Science*, pp. 594–621, Springer, 2023. doi: 10.1007/978-3-031-30872-7_23.

[11] P. Pandare, S. Uniyal, R. Vani, S. Mali, and P. Rumao, "Enhanced password manager using hybrid approach," in *International Conference on Inventive Computation Technologies (ICICT)*, pp. 1793–1798, 2023. doi: 10.1109/ICICT57646.2023.10134398.

[12] Z. Li, W. He, D. Akhawe, and D. Song, "The emperor's new password manager: Security analysis of web-based password managers," in *23rd USENIX Security Symposium*, pp. 465–479, USENIX Association, 2014. `https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/li_zhiwei`.

[13] Y. Fu and D. Wang, "Leaky autofill: An empirical study on the privacy threat of password managers' autofill functionality," in *Annual Computer Security Applications Conference (ACSAC)*, pp. 288–303, 2024. doi: 10.1109/ACSAC63791.2024.00037.

[14] R. Lamarca, P. Beer, and M. Squarcina, "AutoFail: Breaking web boundaries using Android's autofill framework," in *35th USENIX Security Symposium*, pp. 2601–2620, USENIX Association, 2026. `https://www.usenix.org/conference/usenixsecurity26/presentation/lamarca`.

[15] M. Jubur, M. Shirvanian, S. Duraibi, and N. Saxena, "Comparing a store-less password manager with traditional password-only authentication," *IEEE Internet Computing*, vol. 30, pp. 76–86, March/April 2026. doi: 10.1109/MIC.2026.3668165.

[16] E. Halim, T. Dwiangraini, D. Wiryawan, and M. Hebrard, "Implementation of password manager to improve data security for social media account," in *International Conference on Information Management and Technology (ICIMTech)*, pp. 754–759, 2023. doi: 10.1109/ICIMTECH59029.2023.10277793.

[17] S. Rajarajan, P. L. K. Priyadarsini, and G. Vahini, "Design of a secured graphical master password for password manager," in *First International Conference of Advances in Engineering and Computing Technologies for Sustainable Development (AECTSD)*, 2025. doi: 10.1109/AECTSD65988.2025.11410917.

[18] T. Reittinger and G. Pernul, "No password, no problem? a large-scale field study of passkey adoption and usage," in *IEEE Symposium on Security and Privacy (SP)*, pp. 3607–3626, 2026. doi: 10.1109/SP63933.2026.00022.

[19] Microsoft, "Process security and access rights." Microsoft Learn. [Online]. Available: `https://learn.microsoft.com/en-us/windows/win32/procthread/process-security-and-access-rights`. Accessed: Aug. 14, 2026.

[20] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," Tech. Rep. RFC 5869, IETF, May 2010. `https://www.rfc-editor.org/rfc/rfc5869.html`.

[21] Microsoft, "UserConsentVerifier class." Microsoft Learn. [Online]. Available: `https://learn.microsoft.com/en-us/uwp/api/windows.security.credentials.ui.userconsentverifier`. Accessed: Aug. 14, 2026.

[22] Tamarin Prover Team, "Tamarin prover manual: First example." [Online]. Available: `https://tamarin-prover.com/manual/master/book/003_example.html`. Accessed: Aug. 28, 2026.

Appendix A

Installation and User Guide

A.1 Installation and Running the Application

The application source code is in the separate file `Password_Manager_Submission.zip`. Use that archive to install the password manager. This LaTeX package contains the report and supporting evidence. The application's stated requirements are Windows 11 build 22000 or later, 64-bit Python 3.12, 3.13 or 3.14, and Windows Hello set up for the current Windows account.

Extract the application archive and open PowerShell in the folder containing `pyproject.toml`. Run the commands below. The example uses Python 3.14; if you use another supported Python version, change the version number in the first command.

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
.\.venv\Scripts\python.exe -m pip install --no-build-isolation --no-deps -e .
.\.venv\Scripts\python.exe -m password_manager
```

You may need an internet connection during installation to download the required packages. Once these are installed, the local vault functions work offline. To open the application again, run the last command from the project folder. The application is supplied as source code, so this package does not include a standalone Windows installer.

A.2 Creating and Using a Vault

Choose *Create a new encrypted vault*, select a new filename and enter a master passphrase. Use at least 15 characters and follow the checks displayed by the application. Enter the confirmation carefully: spaces and uppercase or lowercase letters make a difference. A forgotten master passphrase cannot be reset. To open a vault that already exists, choose *Open an existing vault* and enter that vault's passphrase.

Once the vault is open, choose *Add credential* to enter demonstration account details or the details you want to store. You can use the password generator to create a password from your chosen character groups. Saving it in the vault does not change the password on the website. Make sure the saved password matches the one used by that account.

The main list shows a summary of each account. Select a record to view, edit, copy, reveal or delete it. Sensitive actions need approval through the broker and Windows Hello. Read the account name and requested action before approving. If you cancel or fail the Windows check, the action does not go ahead. Windows Hello approves the action; you still need the master passphrase to open the vault.

Copying puts the password on the Windows clipboard. After 30 seconds, the application normally tries to clear the copied value if it has not changed. Revealing shows a password for a limited time. Paste a copied password only where you intend to use it. Clearing it later cannot remove a copy already taken by another program or kept in clipboard history.

Choose *Lock vault* when you finish. The default inactivity setting is five minutes; you can change it in *Settings*. While the application is active, `Ctrl+Shift+L` is the emergency lock shortcut. These instructions describe normal use. The main report explains the limits of locking when an attacker controls requests from the GUI.

A.3 Backup and Restore

With the vault open, choose *Create backup*, select a new destination and complete the approval request. Keep the encrypted backup in a suitable location away from the only working copy. Personal vaults should not be stored in the source project folder. The application warns about some known cloud-synchronised locations, but it cannot recognise every way that files may be synchronised.

To restore a backup, go back to the start screen and choose *Restore an encrypted backup*. Select the backup and a new destination for the restored vault. Enter the backup's master passphrase when asked. The application checks the backup before opening the restored copy. Give the restored vault a filename different from both the backup and any working vault. This keeps those other files available if you need them.

A backup holds the vault as it was when the copy was made. An older backup will not contain accounts or password changes added later. It also needs the master passphrase used for that saved version. Restoring brings back that copy; it does not reset a forgotten passphrase. Practise restoring a backup with demonstration data before relying on the backup routine for important accounts, and check backups occasionally.

Appendix B

Additional Test Evidence

B.1 Detailed Test Results

Chapter 6 uses the XML test report included with the final application source. An unchanged copy is included as `evidence/test-results.xml` in this LaTeX package. Table B.1 groups the recorded test cases by module. The file records no failures or errors. The table uses those existing results; the tests were not run again to produce it.

Table B.1: Recorded Test Cases by Module

Module	Cases	Passed	Skipped
`test_activity_guard`	1	1	0
`test_architecture_boundaries`	6	6	0
`test_broker_capabilities`	15	15	0
`test_broker_client`	12	12	0
`test_broker_process`	7	7	0
`test_broker_protocol`	33	33	0
`test_broker_server`	6	6	0
`test_broker_service`	27	27	0
`test_broker_transport`	13	7	6
`test_clipboard_guard`	4	4	0
`test_cloud_locations`	3	3	0
`test_crypto`	30	30	0
`test_file_permissions`	2	1	1
`test_passphrases`	10	10	0
`test_passwords`	9	9	0
`test_project_metadata`	2	2	0
`test_security_audit`	4	4	0
`test_startup`	1	1	0
`test_storage`	41	41	0
`test_storage_validation`	24	24	0
`test_ui`	22	22	0

Continued on the next page

Module	Cases	Passed	Skipped
test_ui_controller	3	3	0
test_unlock_throttle	4	4	0
test_vault_conversion	12	12	0
test_windows_clipboard	1	1	0
test_windows_events	3	3	0
test_windows_hello	14	14	0
test_windows_session	3	3	0
Total	312	305	7

The six transport tests listed below check behaviour specific to Linux. They were skipped because this test run took place on Windows.

```
test_linux_socket_requires_private_runtime_directory
test_missing_and_excessively_long_runtime_paths_are_rejected
test_connect_validates_endpoint_and_authentication_key
test_connect_returns_verified_json_channel
test_connect_closes_connection_when_peer_verification_fails
test_local_pipe_peer_matches_current_user
```

The remaining skipped test is `test_symbolic_link_is_rejected`, in the file-permissions module. The recorded reason is Windows error 1314: the account lacked permission to create the symbolic link needed by the test. A symbolic link points to another file. The skipped test provides no passing result for that check.

To run the tests in future, the project documents these commands:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest
```

Use the supplied test setup with temporary vaults and demonstration accounts. Keep personal vaults separate. Save any new results with their date, environment and source version so they can be distinguished from the evidence here. The development tools were not installed and the tests were not rerun as part of preparing this report.

B.2 Code Quality and Dependency Checks

The evidence directory includes the original `code-quality-dependency-evidence.zip`. It contains the outputs from Ruff, mypy and the dependency-consistency check, along with a record of the tool versions. The recorded summaries read:

```
Ruff: All checks passed!
mypy: Success: no issues found in 39 source files
pip check: No broken requirements found.
```

After installing the development dependencies, these checks can be run from the application project folder:

```
.\.venv\Scripts\python.exe -m ruff check src tests scripts
.\.venv\Scripts\python.exe -m mypy
.\.venv\Scripts\python.exe -m pip check
```

The Ruff configuration enables the E, F, I, B, S and UP rule groups. It documents exceptions for assertions and example password strings in the relevant files. mypy uses strict mode for the `password_manager` package. The reported clean results apply to these settings. Although the project includes settings for measuring code coverage, no completed coverage report was supplied. There is therefore no coverage percentage to report.

B.3 Environment and Tool Versions

Table B.2 lists the available details about the test environment and tools. The installation guide gives the supported Windows version, but the supplied logs do not identify the exact Windows build on which the tests ran.

Table B.2: Recorded Test and Tool Information

Item	Recorded value
Project version	0.8.0 in the final source metadata.
Test timestamp	14 September 2026, 07:31:44.144110, UTC+01:00.
Test duration	21.267 seconds for the recorded suite.
Python	3.14.7.
Ruff	0.16.5.
mypy	1.20.2, compiled installation.
pip	26.2.1 under the project's Python 3.14 environment.
Operating-system build	Not recorded in the supplied tool-version output.

The SHA-256 values below act as fingerprints of the reviewed files. Comparing a later copy's value with the value shown here checks whether the file contains the same bytes. They identify the copies reviewed for this report, but do not prove which source file was used by an earlier test command.

Final application archive:

`Password_Manager_Submission.zip`

```
2c98134413d5dfdd754158edc233a913fe744ddefa144d82e2e6e0dee3229657
```

Automated test report:

`docs/test-results.xml`

```
5a5611942debade066fae2136299d17cdf6aaa2c582cc9f420559eb760ab1f28
```

Code-quality evidence archive:

`code-quality-dependency-evidence.zip`

```
1d7ce2195974a34069209acb8cffc3a8acc0b5f6de0772ccdab0801e8e08c24f
```

Appendix C

Formal Model and Proof Evidence

C.1 Tamarin Model

The final model is in `broker_capability_final.spthy`. It is reproduced below, and an unchanged copy is included in the source package's evidence directory. The model follows one simplified sequence of events for a permission token. It assumes that the broker's stored state remains trusted and does not cover every operation in the application. Section 6.3 explains the rules and what the results tell us.

The listing retains the original comments. The comment above `executability` says that an approved reveal means the safety lemmas are not vacuous. This needs a narrower reading: the result shows that an intended approved release is possible. It does not, by itself, show that every safety property is exercised in all relevant situations. The proof results still depend on the rules and assumptions included in the model.

Its SHA-256 value is:

```
37e48b19aa886d9d749e279cd307569a0441cd0aa82a6081fe0c5fd3a563e417
```

Listing C.1: Final Tamarin Model

```
theory PasswordManagerBrokerFinal
begin

/*
  Symbolic model of one complete Vault Broker permission lifecycle.

  Attacker and trust assumptions
  ------------------------------
  - The visible GUI and IPC messages are controlled by a Dolev-Yao attacker.
    In/Out facts let the attacker observe, alter, block, create, and replay them.
  - The Broker, the Vault Key, and the operating-system process boundary are
    trusted. The key is carried only in private Broker state facts.
  - Broker-owned user approval, including the Windows Hello decision, is
    abstracted by ApproveAndIssueCapability. Windows Hello is not modelled.
  - The capability is fresh and bound to one record, one action, and one unlock
    generation. A linear state fact permits only one outcome: use, expiry, or
    locking.
  - Expiry is a logical transition. This model checks that an expired capability
```

```
    cannot release a password; it does not verify the Python ten-second clock.

  The model deliberately follows a single permission lifecycle. Tamarin may
  instantiate the lifecycle repeatedly, while each instance remains small
  enough for automated proof search.
*/

rule InitializeVault:
  [ Fr(~vault_key), Fr(~generation), Fr(~record), Fr(~password) ]
  --[ VaultInitialized(~vault_key, ~generation),
      CredentialAvailable(~generation, ~record, ~password) ]->
  [ ReadyForRequest(~generation, ~vault_key, ~record, ~password),
    Out(~record) ]

rule RequestAction:
  [ In(<record, action>) ]
  --[ Requested(record, action) ]->
  [ Pending(record, action) ]

rule ApproveAndIssueCapability:
  [ ReadyForRequest(generation, vault_key, record, password),
    Pending(record, action),
    Fr(~capability) ]
  --[ Approved(record, action),
      CapabilityIssued(~capability, record, action, generation) ]->
  [ CapabilityReady(
      ~capability, record, action, generation, vault_key, password
    ),
    Out(~capability) ]

rule UseCapability:
  [ CapabilityReady(
      capability, record, action, generation, vault_key, password
    ),
    In(<capability, record, action>) ]
  --[ SecretReleased(capability, record, action, password, generation),
      CapabilityUsed(capability) ]->
  [ ReleasedState(generation, vault_key),
    Out(password) ]

rule ExpireCapability:
  [ CapabilityReady(
      capability, record, action, generation, vault_key, password
    ) ]
  --[ CapabilityExpired(capability, generation) ]->
  [ ExpiredState(generation, vault_key) ]

rule LockBeforeCapability:
  [ ReadyForRequest(generation, vault_key, record, password),
    In('lock') ]
  --[ VaultLocked(generation) ]->
  [ LockedState(generation, vault_key) ]

rule LockWithCapability:
  [ CapabilityReady(
```

```
      capability, record, action, generation, vault_key, password
    ),
    In('lock') ]
  --[ VaultLocked(generation) ]->
  [ LockedState(generation, vault_key) ]

rule LockAfterExpiry:
  [ ExpiredState(generation, vault_key), In('lock') ]
  --[ VaultLocked(generation) ]->
  [ LockedState(generation, vault_key) ]

rule LockAfterRelease:
  [ ReleasedState(generation, vault_key), In('lock') ]
  --[ VaultLocked(generation) ]->
  [ LockedState(generation, vault_key) ]

/* A complete approved reveal exists, so the safety lemmas are not vacuous. */
lemma executability:
  exists-trace
  "Ex capability record password generation vault_key #init #request
      #issue #release.
      VaultInitialized(vault_key, generation) @ init
    & CredentialAvailable(generation, record, password) @ init
    & Requested(record, 'reveal_password') @ request
    & CapabilityIssued(
        capability, record, 'reveal_password', generation
      ) @ issue
    & SecretReleased(
        capability, record, 'reveal_password', password, generation
      ) @ release
    & init < request & request < issue & issue < release"

/* Every password release is preceded by approval for its record and action. */
lemma approval_required_before_release:
  "All capability record action password generation #release.
      SecretReleased(
        capability, record, action, password, generation
      ) @ release
    ==> (Ex #approval.
          Approved(record, action) @ approval
        & approval < release)"

/* The released password and scope match the exact issued capability. */
lemma release_matches_issued_scope_and_password:
  "All capability record action password generation #release.
      SecretReleased(
        capability, record, action, password, generation
      ) @ release
    ==> (Ex #issue #available.
          CapabilityIssued(
            capability, record, action, generation
          ) @ issue
        & CredentialAvailable(
            generation, record, password
          ) @ available
```

```
        & issue < release
        & available < release)"

/* An expired capability cannot release a password. If the same generation is
   eventually locked after a release, that release must precede the lock. */
lemma expiry_and_lock_prevent_late_release:
  "(All capability record action password generation #expired #release.
       CapabilityExpired(capability, generation) @ expired
     & SecretReleased(
         capability, record, action, password, generation
       ) @ release
     ==> F)
   &
   (All capability record action password generation #locked #release.
       VaultLocked(generation) @ locked
     & SecretReleased(
         capability, record, action, password, generation
       ) @ release
     ==> release < locked)"

/* A single capability cannot produce two distinct release events. */
lemma no_capability_replay:
  "All capability record1 record2 action1 action2 password1 password2
       generation1 generation2 #i #j.
      SecretReleased(
        capability, record1, action1, password1, generation1
      ) @ i
    & SecretReleased(
        capability, record2, action2, password2, generation2
      ) @ j
    ==> #i = #j"

/* The attacker never learns a Vault Key generated inside the trusted Broker. */
lemma vault_key_secrecy:
  "All vault_key generation #created #known.
      VaultInitialized(vault_key, generation) @ created
    & K(vault_key) @ known
    ==> F"

end
```

C.2 Proof Commands and Tool Versions

Each recorded proof output comes from a separate command checking one lemma. The Bash commands below check the same lemmas and use the same output filenames. Run them from a directory containing the model, with Tamarin and Maude already installed. The commands make the checks repeatable; they were not run again while this report was being prepared.

```
set -o pipefail
tamarin-prover broker_capability_final.spthy \
  --prove=executability 2>&1 | tee tamarin-executability-output.txt
tamarin-prover broker_capability_final.spthy \
  --prove=approval_required_before_release 2>&1 | tee tamarin-approval-output.txt
tamarin-prover broker_capability_final.spthy \
  --prove=release_matches_issued_scope_and_password 2>&1 | tee tamarin-scope-output.txt
tamarin-prover broker_capability_final.spthy \
  --prove=expiry_and_lock_prevent_late_release 2>&1 | tee tamarin-expiry-lock-output.txt
tamarin-prover broker_capability_final.spthy \
  --prove=no_capability_replay 2>&1 | tee tamarin-replay-output.txt
tamarin-prover broker_capability_final.spthy \
  --prove=vault_key_secrecy 2>&1 | tee tamarin-vault-key-output.txt
```

The version record lists Tamarin 1.12.0 and Maude 3.5.1. For the Tamarin tool build, it gives Git revision `82780bbaf3328a45f624ddb41e51bf75425f851c` on the master branch and reports uncommitted changes. The recorded compilation time is 7 March 2026, 14:22:39 UTC. These details matter when comparing runs. The evidence does not identify the uncommitted changes.

C.3 Individual Proof Results

Table C.1 lists the exact lemma names and the processing time recorded for each run. The times belong to these separate commands; they are not a general performance ranking of the properties. Each selected lemma is verified in its detailed output. The logs also confirm that the model passed the checks for being well formed.

Table C.1: Individual Targeted Proof Runs

Selected lemma	Steps	Time (s)
`executability`	7	1.02
`approval_required_before_release`	3	0.81
`release_matches_issued_scope_and_password`	3	0.95
`expiry_and_lock_prevent_late_release`	18	1.04
`no_capability_replay`	8	1.00
`vault_key_secrecy`	2	0.81

The supporting package contains the original `tamarin-final-evidence.zip`, with the model, version output, all six individual logs and a combined summary. The full logs allow the results to be checked in more detail than a screenshot or filtered list. A run may show “analysis incomplete” for a lemma that was not selected in that command. Read this alongside the separate run in which that lemma was selected and completed.

The SHA-256 value of the original evidence archive is:

```
dd6361d6d3989228a89bb599d662501dea628e582b519cc0a4b506f1245e8b55
```